AI 소프트웨어 기본설계서

CareerSync

버전: 1.0
작성일: 2026-10-11
작성자:

20240232_응오 후인 느 (조장)

20240233_응웬치롱

20203051_이건호(부조장)

1. 프로젝트 개요 (Introduction)

1.1 개발 목적

CareerSync 는 인공지능(AI) 기술을 활용하여 구직자와 기업 간의 채용 정보 불일치를 줄이고, 효율적인 취업 준비 및 채용 관리를 지원하는 웹 기반 플랫폼이다.

기존의 채용 플랫폼에서는 구직자가 수많은 채용공고를 직접 확인하고 자신의 역량과 채용 조건을 비교해야 하는 어려움이 있다. 또한 기업은 지원자의 이력서와 채용 요건을 개별적으로 검토해야 하므로 많은 시간과 인력이 소요된다.

본 프로젝트는 이러한 문제를 해결하기 위해 구직자의 이력서와 기업의 채용공고를 AI 로 분석하고, 직무 역량 및 요구 조건을 비교하여 적합도와 추천 근거를 제공하는 것을 목표로 한다.

이를 통해 구직자는 자신의 역량에 적합한 채용공고를 효율적으로 탐색하고 부족한 역량을 파악할 수 있으며, 기업은 채용 조건에 적합한 지원자를 체계적으로 관리할 수 있다.

1.2 개발 범위

CareerSync 는 구직자와 기업 사용자를 대상으로 하는 웹 기반 서비스로 설계한다.

(1) 구직자 서비스

- 회원가입 및 로그인
- 개인 프로필 및 이력서 관리

- 채용공고 검색 및 상세 정보 조회
- AI 기반 이력서 분석
- 채용공고와 이력서 간 적합도 분석
- 맞춤형 채용공고 추천
- 채용공고 지원 및 지원 현황 관리

(2) 기업 서비스

- 기업 회원가입 및 로그인
- 기업 정보 관리
- 채용공고 등록 및 관리
- 지원자 목록 및 이력서 조회
- AI 기반 지원자 적합도 분석
- 지원자 평가 및 채용 진행 상태 관리

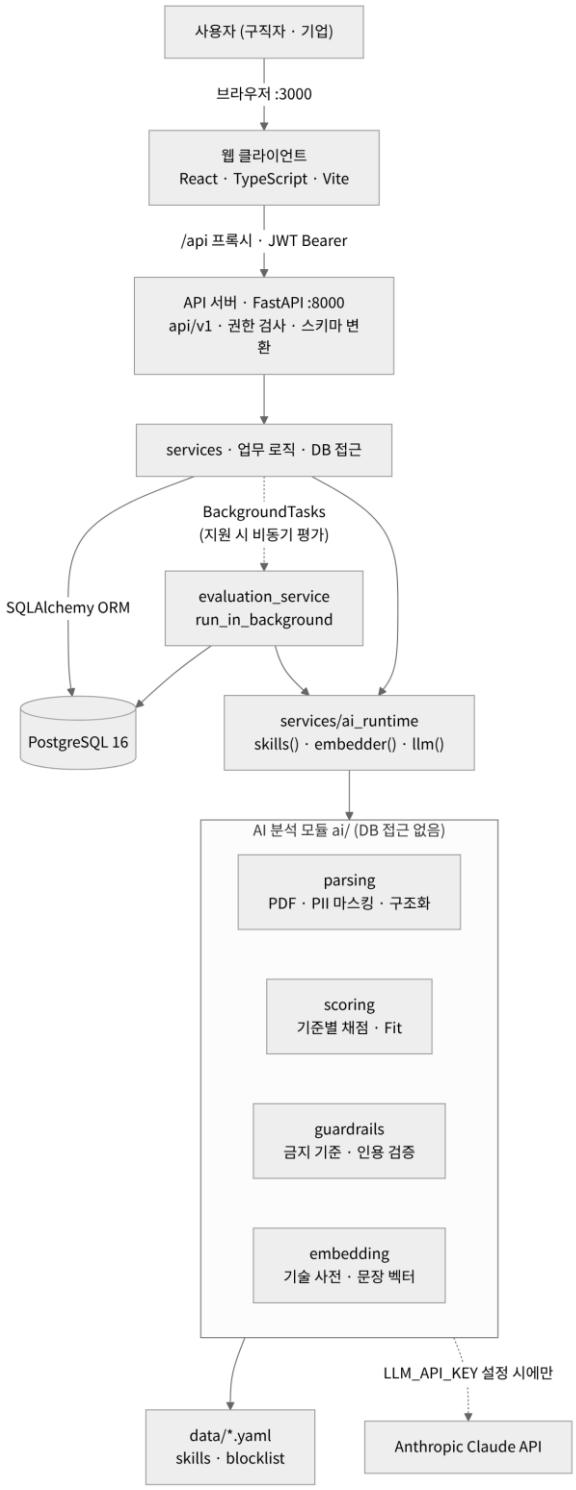
(3) AI 분석 서비스

- 이력서 및 채용공고 텍스트 분석
- 직무 관련 기술 및 요구 역량 추출
- 구직자 역량과 채용 요건 비교
- 적합도 점수 및 추천 근거 생성
- 부족한 역량 분석 및 개선 방향 제시

2. AI 시스템 아키텍처 (System Architecture)

2.1 전체 구조도

[그림 2-1] CareerSync 시스템 구성도



CareerSync 는 웹 클라이언트, API 서버, 업무 로직(services), AI 분석 모듈, PostgreSQL 데이터베이스로

구성된다. 웹 클라이언트는 /api 프록시를 통해서만 서버와 통신하며, API 계층(api/v1)은 인증·권한(CurrentUser, CompanyUser)을 확인하고 요청·응답 스키마를 변환한다. 업무 로직과 DB 접근은 services 계층이 담당하고, AI 기능은 services/ai_runtime.py 를 거쳐 ai/ 모듈을 호출한다. ai/ 모듈은 데이터베이스에 접근하지 않으며 app.* 코드를 참조하지 않는다.

이력서 분석 흐름 (동기 처리)

POST /api/cv 요청 안에서 PDF 검증 → 텍스트 추출 → 개인정보 마스킹 → structure_cv() 구조화 → cv_documents·cv_profiles·cv_* 저장까지 수행한 뒤 결과(CvOut)를 응답한다.

지원자 평가 흐름 (비동기 처리)

입사 지원 요청은 applications 와 evaluations(pending)만 저장하고 즉시 응답한다. 기준별 채점은 FastAPI BackgroundTasks 로 같은 서버 프로세스에서 evaluation_service.run_in_background()가 수행하며, 결과를 criterion_scores 에 저장하고 evaluations 상태를 done 또는 failed 로 변경한다. Fit 은 기업이 지원자 목록을 조회할 때 저장된 점수와 현재 가중치로 계산한다.

- Client: React·TypeScript 및 Vite 기반 웹 클라이언트
- Server: Python FastAPI 기반 REST API 서버(/api)
- Database: PostgreSQL 16 기반 관계형 데이터베이스, SQLAlchemy ORM 및 Alembic 마이그레이션

2.2 기술 스택

- Language: Python(백엔드·AI), TypeScript(프런트엔드)
- Framework: FastAPI, React(Vite), SQLAlchemy, Pydantic / AI 처리 라이브러리: PyMuPDF, sentence-transformers 등
- Infrastructure: Docker Compose 기반 개발 환경(PostgreSQL 16·백엔드·프런트엔드 컨테이너)

API 서버 ↔ AI 모듈 호출 방식

- services 는 services/ai_runtime.py 의 skills(), embedder(), llm()을 통해서만 AI 구성요소를 얻는다. 테스트에서는 이 지점을 가짜 구성요소(FakeLLM, hashing 임베더)로 교체한다.
- AI 모듈에는 DB 모델이 아닌 일반 객체(Candidate, Criterion, JobContext)를 전달한다. 채점에

사용하는 평가기준은 criteria_service.load_for_scoring()으로만 읽는다.

- 예외: 채용공고 기준 제안(POST /api/jobs/{job_id}/analyze)은 api/v1/jobs.py 가 ai.parsing.extract.analyze_jd 를 직접 호출한다.

외부 AI 서비스 연동 범위

기능	외부 호출(Claude)	호출 빈도
이력서 구조화 (structure_cv)	LLM_API_KEY 설정 시 사용, 없으면 규칙 기반	업로드 1 건당 1 회
채용공고 기준 제안 (analyze_jd)	LLM_API_KEY 설정 시 사용, 없으면 규칙 기반	요청 1 회당 1 회
project · other 기준 채점	LLM_API_KEY 설정 시 사용, 없으면 검색 방식	기준 1 개·지원 1 건당 1~2 회
skill · experience · education · certificate 채점	사용하지 않음 (규칙·사전·임베딩)	—
문장 임베딩	사용하지 않음 (서버 내 로컬 모델)	—

분석 실패 시 처리 절차

- API 키가 없으면 get_llm_client()가 None 을 반환하고 모든 기능이 규칙·검색 방식으로 동작한다 (개발 규칙: 모든 기능은 LLM_API_KEY 없이도 동작해야 함).
- LLM 채점 결과의 인용문이 마스킹된 이력서에 존재하지 않으면 1 회 재시도하고, 그래도 실패하면 키워드·임베딩 검색 방식으로 채점한다.
- 평가 작업 전체가 실패하면 evaluations.status=failed 와 error 를 저장한다. 기업 담당자는 '다시 분석'(POST /api/applications/{application_id}/evaluation/retry)으로 재평가할 수 있다.
- PDF 검증 실패는 422 오류 코드(CV_TOO_LARGE, CV_NOT_PDF, CV_UNREADABLE, CV_ENCRYPTED, CV_TOO_MANY_PAGES, CV_NO_TEXT)로 즉시 안내한다.

3. 기능 설계 (Functional Design)

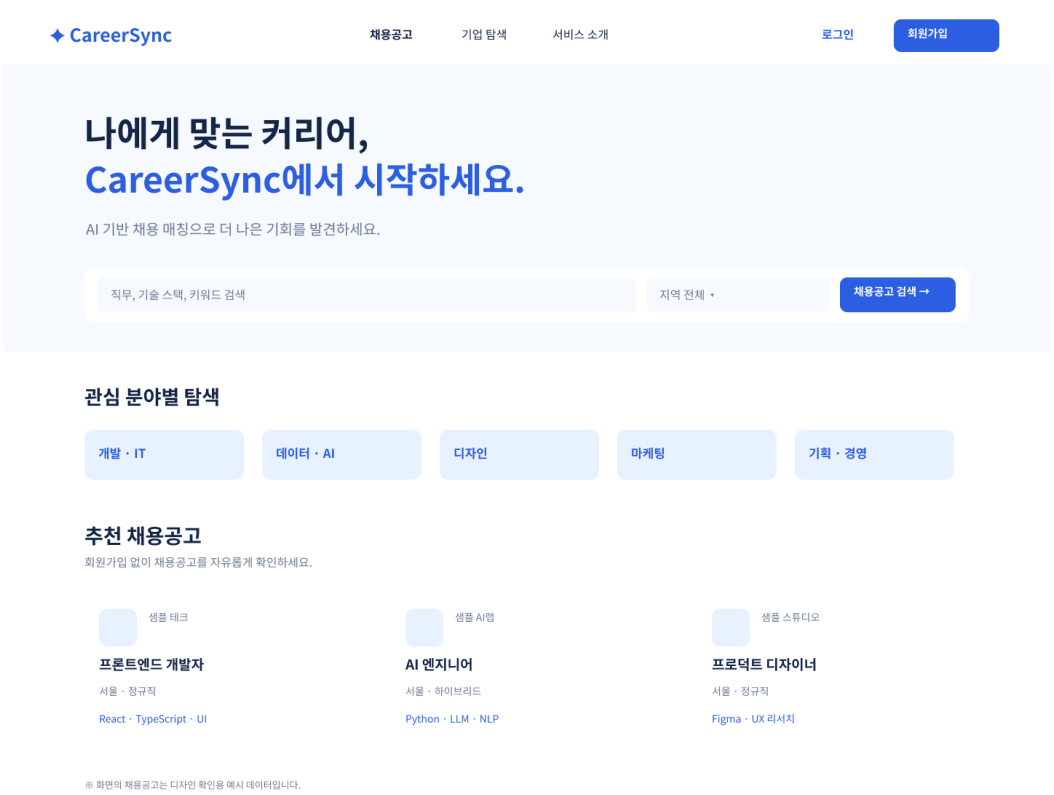
3.1 주요 기능 목록

CareerSync 는 구직자와 기업을 연결하는 AI 기반 채용 및 취업 준비 관리 플랫폼으로, 사용자

관리, 채용공고 관리, 이력서 분석, AI 적합도 분석 및 입사 지원 관리 기능을 중심으로 구성된다.사용자 유형에 따라 구직자와 기업의 기능을 구분하며, 각 기능은 웹 인터페이스를 통해 제공하도록 설계한다.

[표 3-1] CareerSync 주요 기능 목록

기능 ID	기능명	주요 내용	이용 대상
F-01	사용자 관리	회원가입, 로그인, 계정 유형 선택 및 권한 관리	공통
F-02	프로필 관리	구직자 희망 근무 지역·고용 형태·직무 키워드 설정, 인재풀 공개 동의 및 기업 정보 등록·수정	구직자 / 기업
F-03	채용공고 탐색	채용공고 검색, 필터링 및 상세 정보 조회	구직자 / 비회원
F-04	이력서 관리	이력서 등록, 정보 확인 및 AI 분석 요청	구직자
F-05	AI 적합도 분석	이력서와 채용공고의 요구 역량 및 선호조건 비교하여 구직자 맞춤 공고 추천, 기업 맞춤 인재 추천 및 정확도 분석 결과 제공	구직자 / 기업
F-06	입사 지원 관리	채용공고 지원 및 지원 진행 상태 확인	구직자
F-07	채용공고 관리	기업의 채용공고 등록 및 관리	기업
F-08	지원자 관리	지원자 목록 조회, 이력서 확인 및 채용 상태 관리	기업



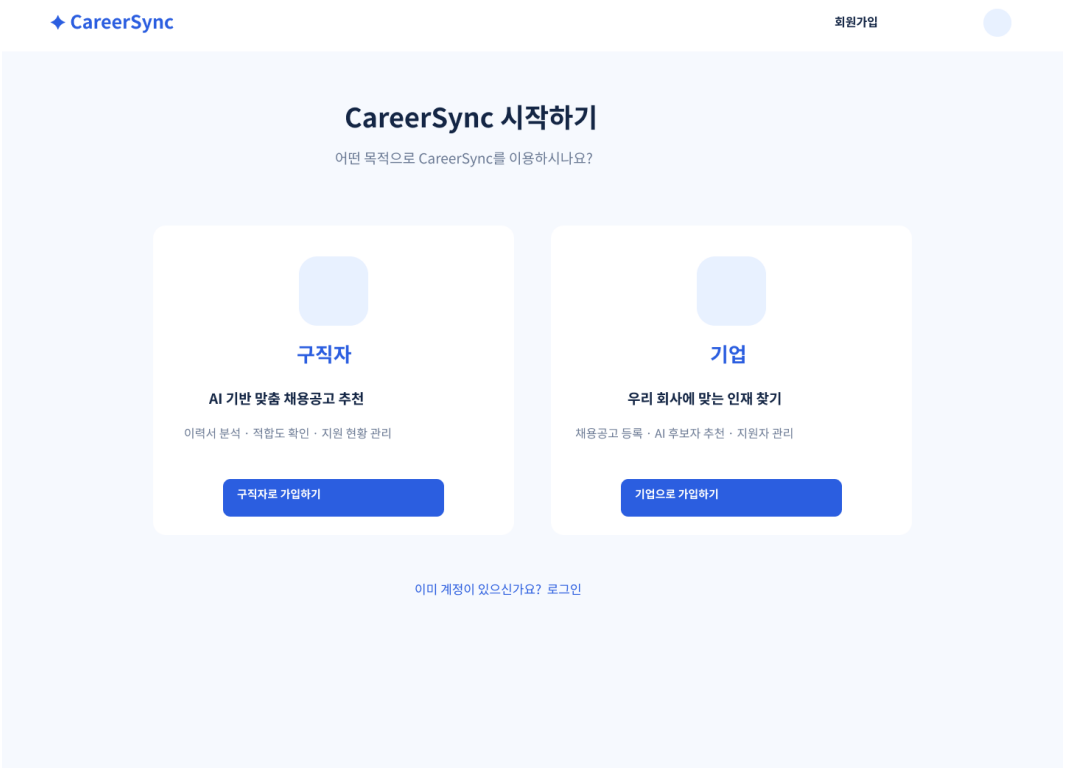
[그림 3-1] CareerSync 메인 랜딩 페이지

(1) 사용자 관리 기능

사용자 관리 기능은 구직자와 기업 사용자의 계정을 생성하고, 계정 유형에 따라 서비스 접근 권한을 구분하는 기능이다.

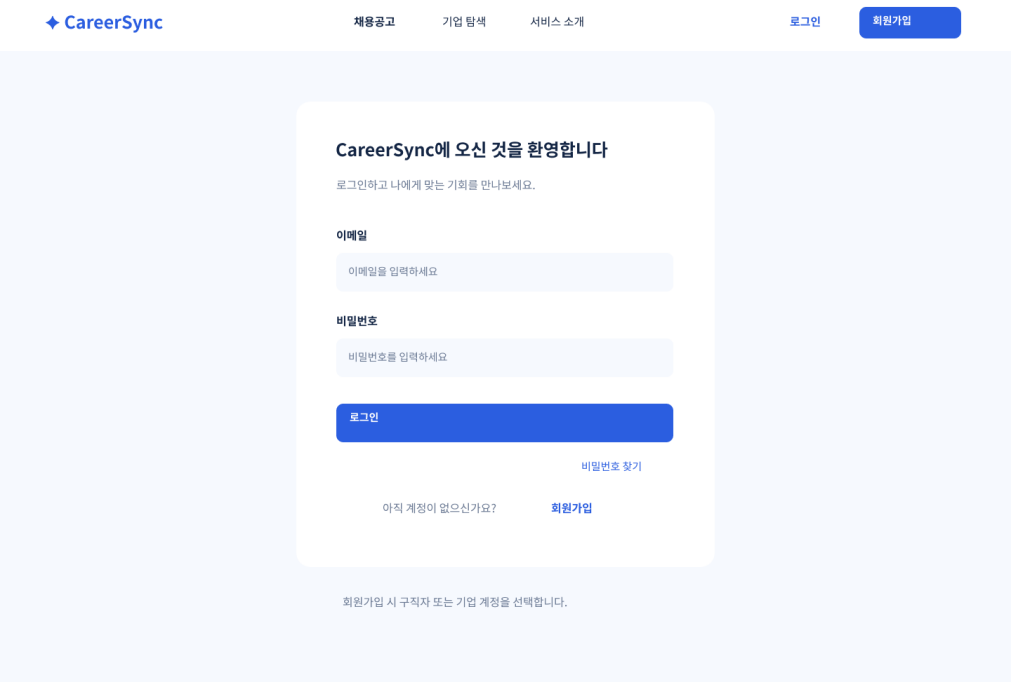
사용자는 회원가입 과정에서 구직자 또는 기업 계정 유형을 선택한다. 구직자는 이력서 분석 및 채용공고 지원 기능을 이용하며, 기업은 채용공고 등록과 지원자 관리 기능을 이용하도록 설계한다.

[그림 3-2] 계정 유형 선택 화면

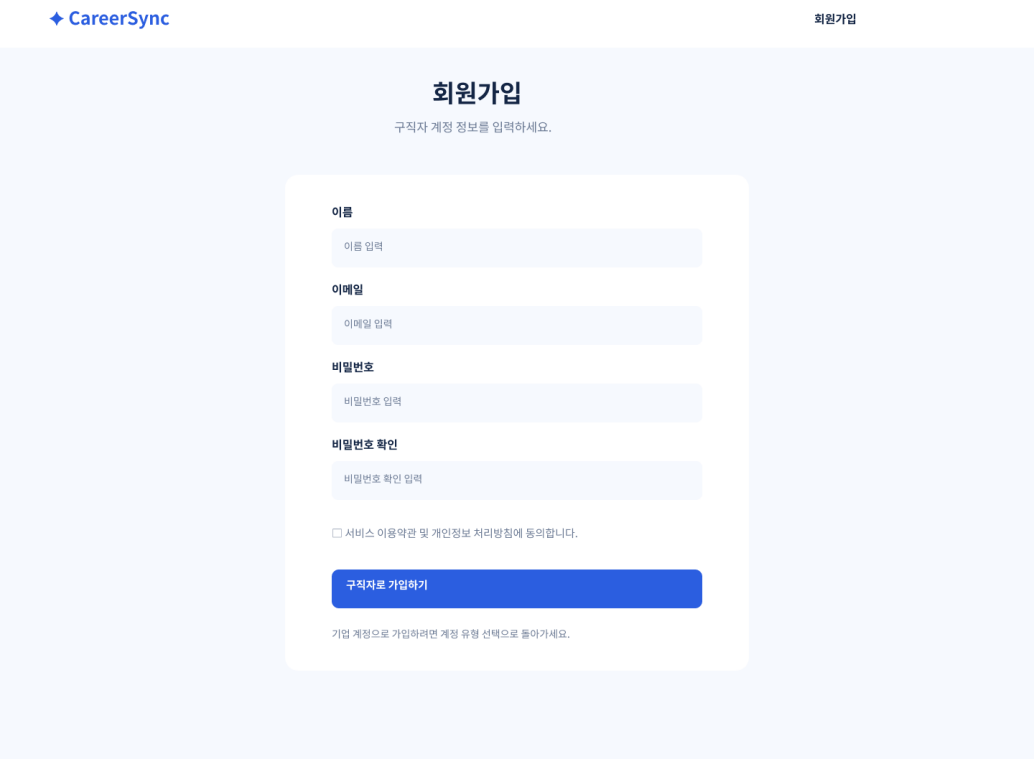


계정 유형을 선택한 사용자는 해당 유형의 회원가입 화면으로 이동한다. 구직자 회원가입 화면에서는 이름, 이메일, 비밀번호 및 비밀번호 확인 정보를 입력하고 서비스 이용약관과 개인정보 처리방침에 동의하도록 설계한다.

[그림 3-3] 로그인 화면



[그림 3-4] 회원가입 정보 입력 화면



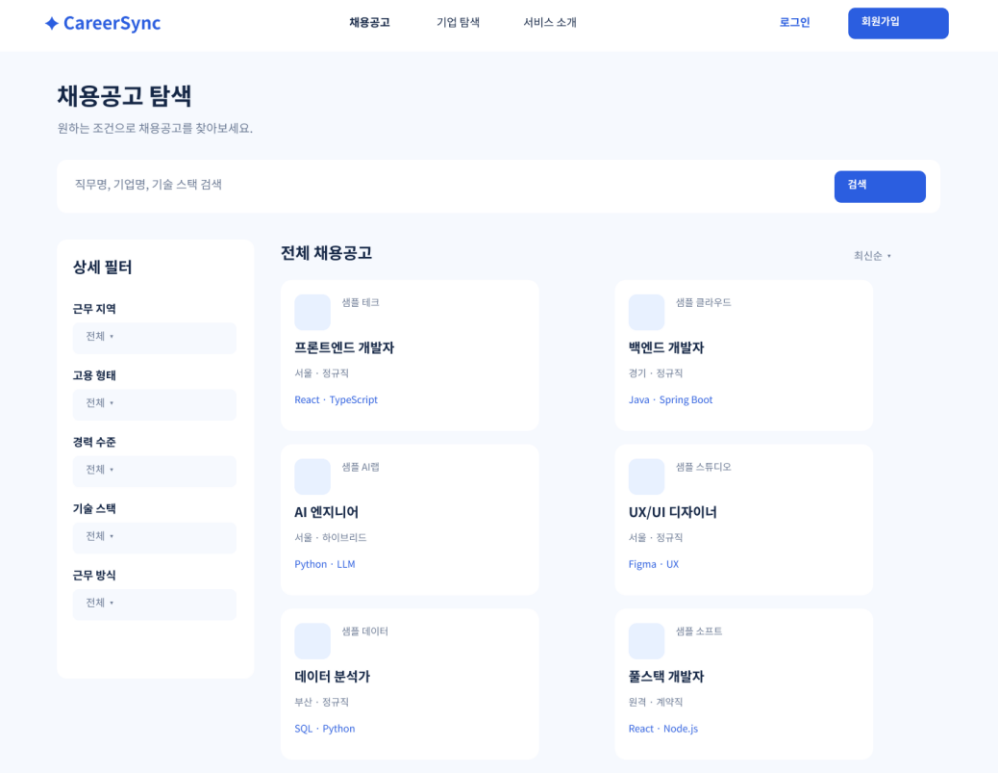
회원가입을 완료한 사용자는 로그인 화면에서 이메일과 비밀번호를 입력하여 인증을 진행한다. 로그인 성공 시 사용자 유형에 맞는 대시보드로 이동하도록 설계한다.

(2) 채용공고 탐색 기능

채용공고 탐색 기능은 사용자가 다양한 기업의 채용 정보를 검색하고 자신에게 적합한 채용공고를 확인할 수 있도록 지원한다.

사용자는 직무명, 기업명 및 기술 스택을 기준으로 채용공고를 검색할 수 있다. 또한 근무 지역, 고용 형태, 경력 요건 등의 조건을 활용하여 검색 결과를 필터링하도록 설계한다.

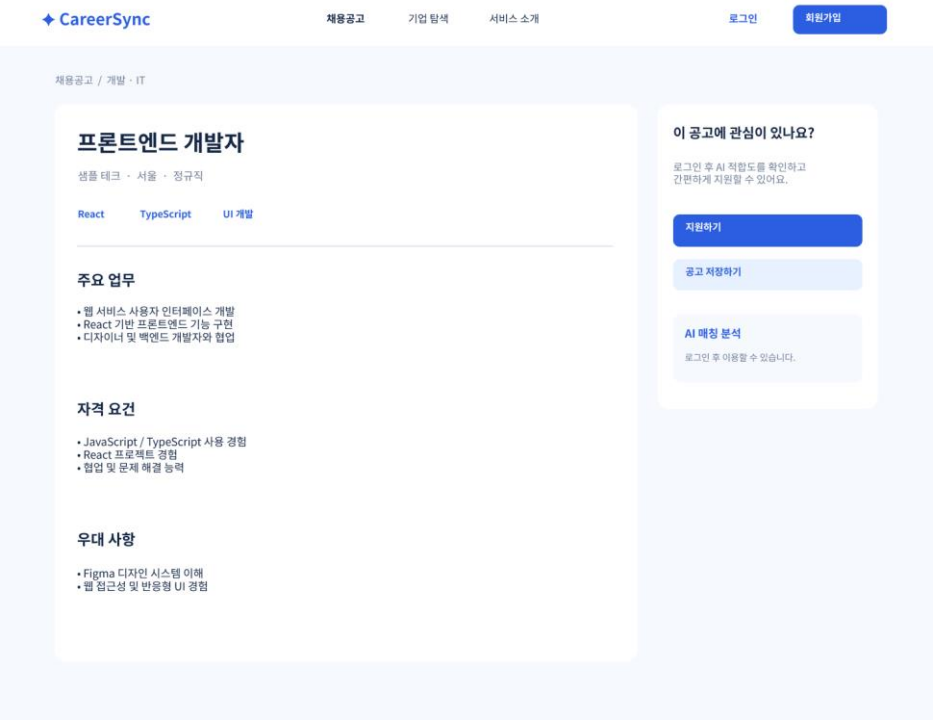
[그림 3-5] 채용공고 탐색 화면



채용공고를 선택하면 상세 화면에서 기업명, 직무명, 주요 업무, 자격 요건 및 우대 사항 등을 확인할 수 있다.

비회원도 채용공고의 상세 내용을 조회할 수 있지만, 지원하기, 공고 저장 및 AI 매칭 분석 등 개인화 기능은 로그인 후 이용하도록 설계한다.

[그림 3-6] 채용공고 상세 화면

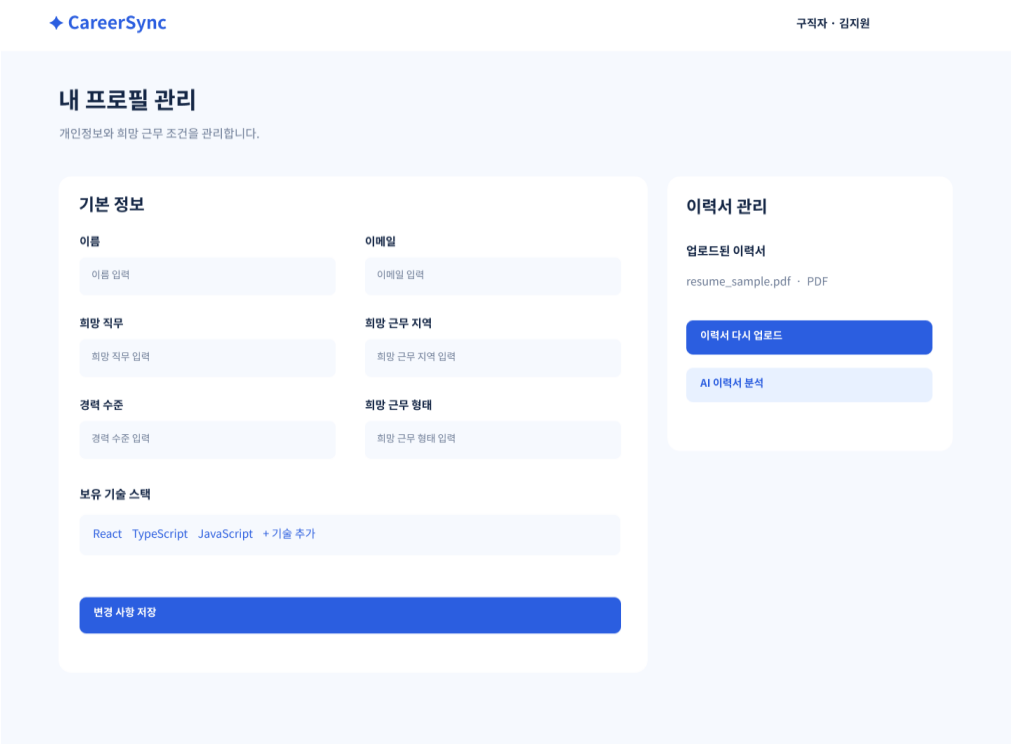


(3) 구직자 프로필 및 AI 이력서 분석 기능

구직자는 프로필 관리 화면에서 이름, 이메일, 희망 직무, 희망 근무 지역, 경력 수준, 희망 근무 형태 및 보유 기술 스택을 등록하거나 수정할 수 있다.또한 PDF 형식의 이력서를 업로드하고 등록된 이력서를 확인하거나 새로운 파일로 변경할 수 있도록 설계한다.

PDF 이력서를 업로드하면 서버는 문서에서 텍스트를 추출하고 개인정보를 마스킹한다. 문서 메타데이터와 마스킹된 텍스트는 cv_documents 에 저장하고, 분석된 기술·경력·프로젝트·학력·자격증 정보는 이력서 버전과 연결된 관계형 테이블에 나누어 저장한다. 원본 PDF 바이너리는 데이터베이스에 보관하지 않는다.

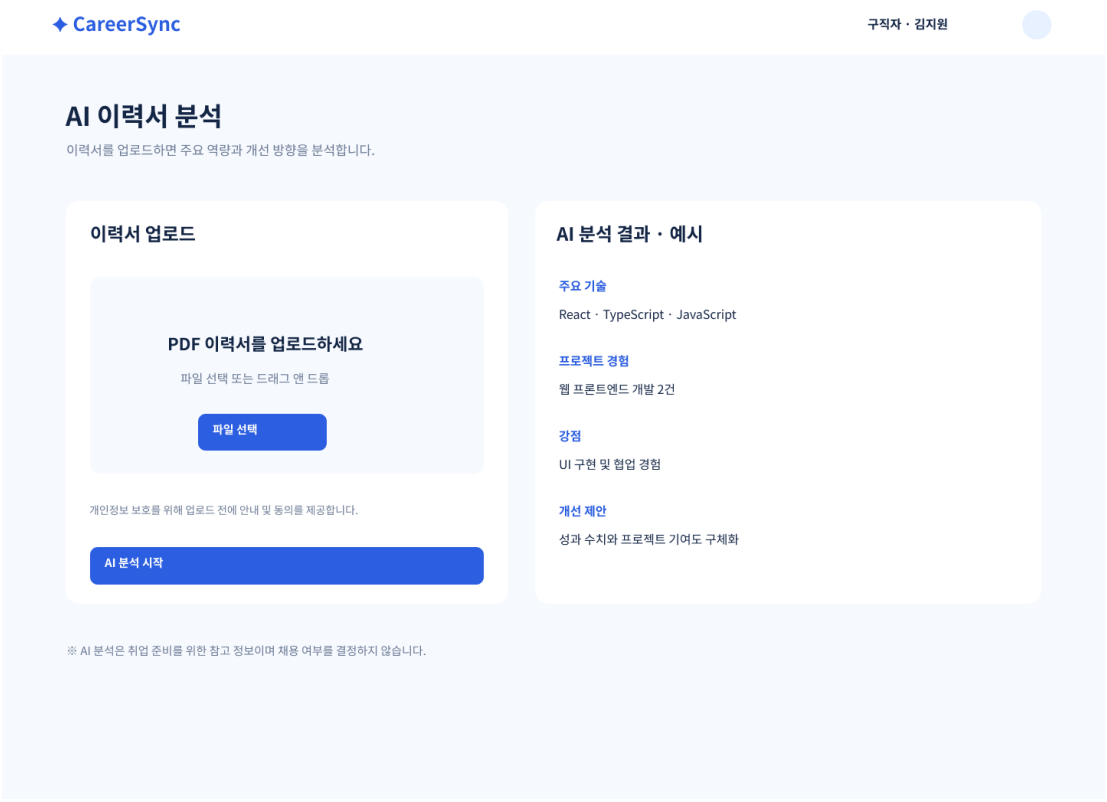
[그림 3-7] 구직자 프로필 관리 화면



AI 이력서 분석 기능은 업로드된 이력서에서 직무 관련 주요 정보를 추출하여 구직자가 자신의 역량을 파악할 수 있도록 지원한다.

분석 결과에는 주요 기술, 프로젝트 경험, 강점 및 개선 제안 등을 제공하도록 설계한다. 이를 통해 구직자는 이력서의 보완이 필요한 부분을 확인하고 취업 준비에 활용할 수 있다.

[그림 3-8] AI 이력서 분석 화면



(4) AI 채용 적합도 분석 기능

AI 채용 적합도 분석 기능은 구직자의 이력서 및 프로필 정보와 기업 채용공고의 요구 조건을 비교하여 직무 관련 적합도를 제공하는 기능이다.

구직자 대시보드에서는 등록된 이력서 상태, 추천 채용공고, 지원 현황 및 저장한 공고를 한눈에 확인할 수 있도록 구성한다.

또한 이력서와 희망 조건을 기반으로 추천 채용공고 목록과 적합도 정보를 표시하도록 설계한다.

[그림 3-9] 구직자 대시보드 화면

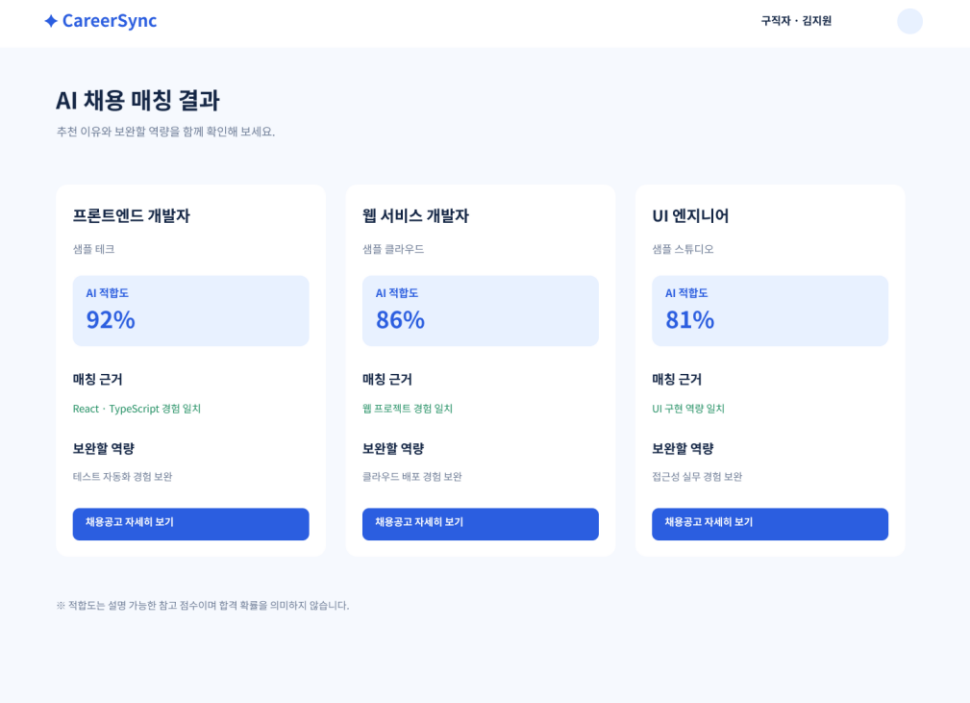


AI 매칭 결과 화면에서는 각 채용공고에 대한 적합도 점수와 추천 근거를 제공한다.

분석 결과에는 구직자의 보유 기술 및 프로젝트 경험과 채용공고 요구 조건 사이의 일치 항목, 보완이 필요한 역량 등을 포함하도록 설계한다.

이를 통해 구직자는 채용공고별 자신의 강점과 부족한 역량을 파악하고, 지원 여부를 판단하거나 취업 준비 방향을 설정하는 데 참고할 수 있다.

[그림 3-10] AI 채용 매칭 결과 화면



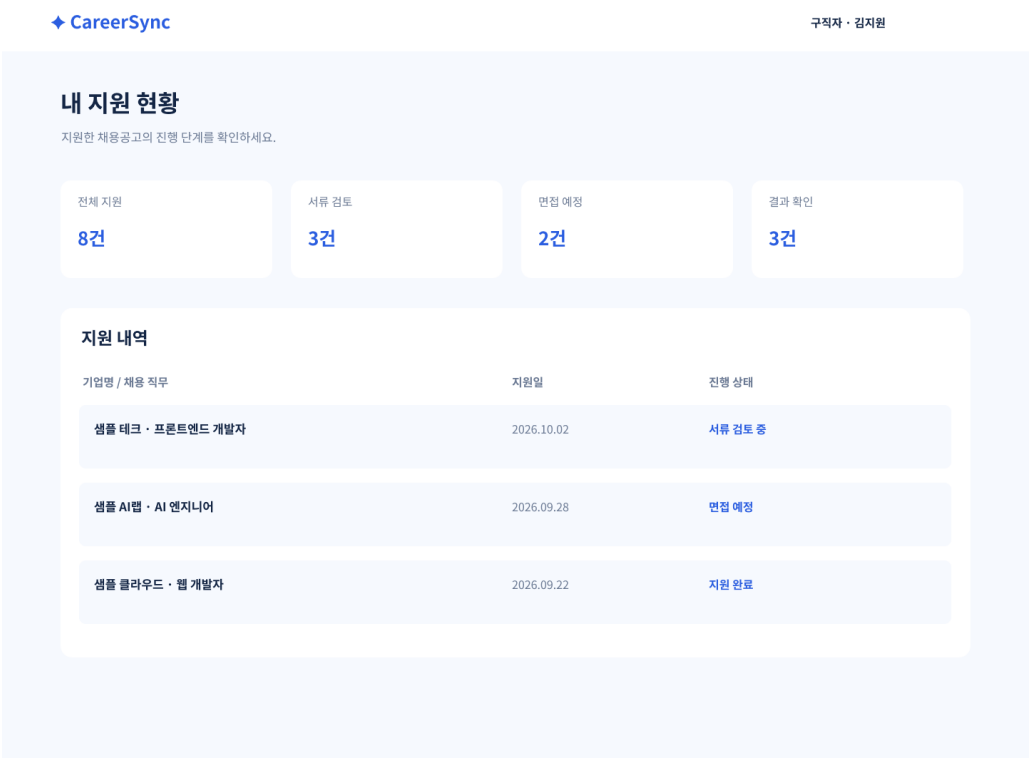
(5) 입사 지원 및 지원 현황 관리 기능

구직자는 채용공고 상세 화면에서 입사 지원을 진행할 수 있으며, 지원한 채용공고의 진행 상태를 별도로 확인할 수 있도록 설계한다.

지원 현황 관리 화면에서는 지원 기업, 채용 직무, 지원 날짜 및 현재 진행 상태를 제공한다.

이를 통해 사용자는 여러 기업의 지원 내역을 통합적으로 관리하고 채용 진행 상황을 쉽게 파악할 수 있다.

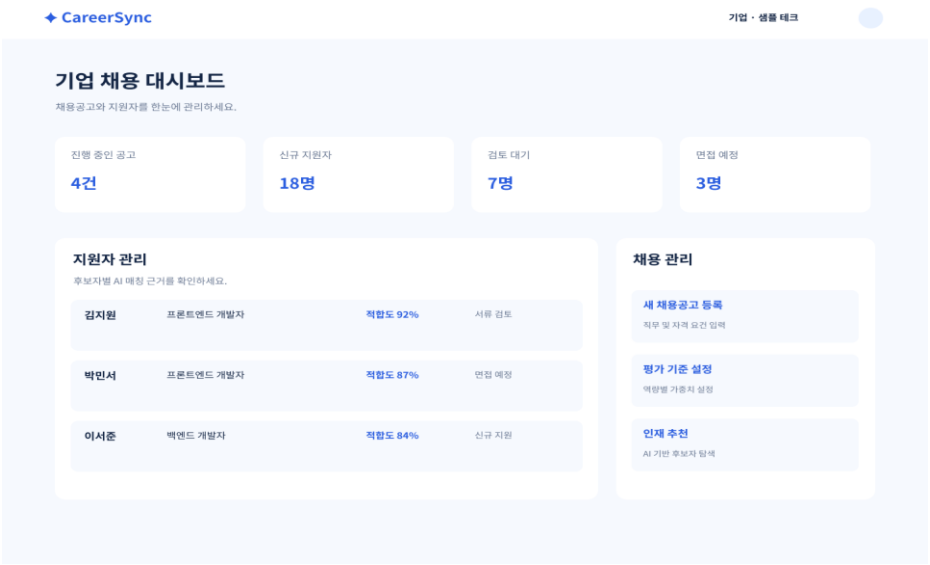
[그림 3-11] 구직자 지원 현황 관리 화면



(6) 기업 채용공고 및 지원자 관리 기능

기업 사용자는 기업 전용 대시보드를 통해 진행 중인 채용공고, 신규 지원자, 검토 대기 지원자 및 면접 예정 인원 등의 정보를 확인할 수 있도록 설계한다. 또한 지원자 관리, 채용공고 등록, 평가 기준 설정 및 AI 기반 인재 추천 기능에 접근할 수 있도록 구성한다.

[그림 3-12] 기업 채용 대시보드 화면



기업은 채용공고 등록 화면에서 채용 직무명, 근무 지역, 고용 형태, 경력 요건, 주요 업무, 자격 요건 및 우대 사항을 입력할 수 있다.

등록된 채용 조건은 구직자에게 채용공고로 제공되며, AI 적합도 분석의 비교 기준으로 활용하도록 설계한다.

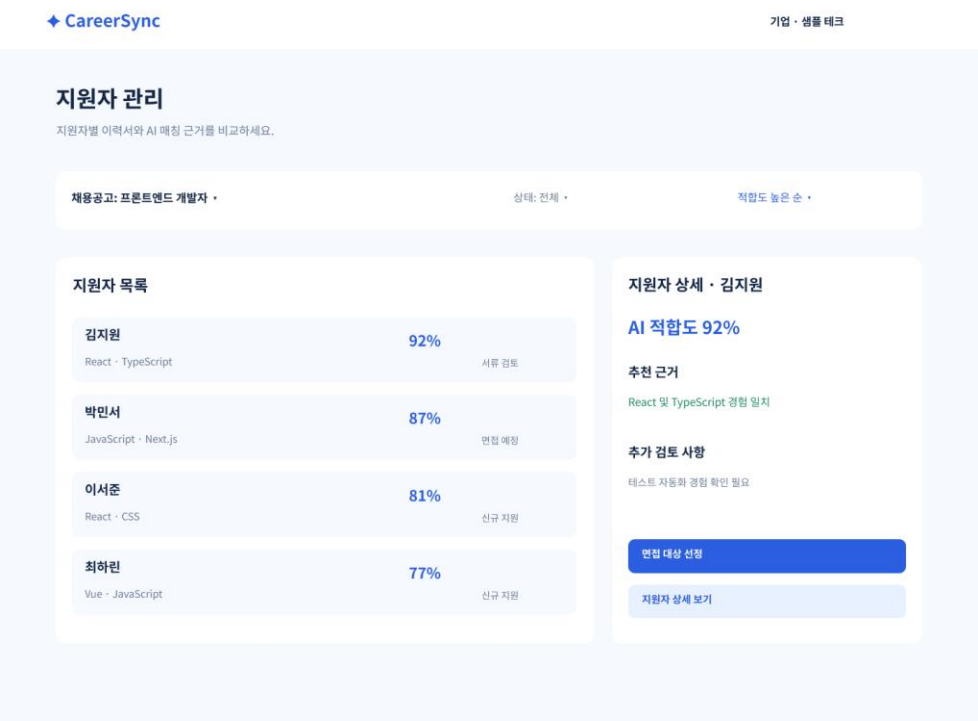
[그림 3-13] 기업 채용공고 등록 화면



기업 지원자 관리 기능은 특정 채용공고에 지원한 지원자 목록을 조회하고, 각 지원자의 보유 기술, AI 적합도 및 채용 진행 상태를 확인할 수 있도록 지원한다.

기업 담당자는 지원자별 상세 정보와 추천 근거를 검토하고 면접 대상자를 선정하는 등 채용 절차를 관리할 수 있도록 설계한다.

[그림 3-14] 기업 지원자 관리 화면



※ 본 절에 제시된 화면은 Figma 를 활용하여 제작한 UI 설계안이다. 화면에 표시된 기업, 채용공고, 지원자, 통계 및 AI 적합도 수치는 기능 설명을 위한 예시 데이터이며, 실제 분석 성능이나 운영 결과를 의미하지 않는다.

3.2 프로세스 흐름도 (Flowchart)

CareerSync 의 주요 서비스 처리 과정은 구직자의 채용공고 탐색 및 입사 지원 프로세스, AI 기반 채용 적합도 분석 프로세스, 기업의 채용공고 등록 및 지원자 관리 프로세스로 구분한다.

또한 비회원도 채용공고를 자유롭게 탐색할 수 있도록 설계하여 사용자 접근성을 높인다. 각 프로세스의 주요 처리 단계와 조건 분기를 흐름도로 나타낸다.

(1) 전체 사용자 이용 흐름

CareerSync 는 구직자와 기업의 이용 목적에 따라 서로 다른 서비스 흐름을 제공한다.

구직자는 프로필 생성, 이력서 업로드 및 역량 추출, AI 적합도 분석, 맞춤 채용공고 탐색, 입사 지원 및 진행 현황 확인 과정을 수행한다.

기업은 기업 프로필 생성, 채용공고 및 자격 요건 등록, 지원자 평가 기준 설정, AI 추천 지원자 확인, 후보자 선정 및 연락 과정을 수행하도록 설계한다.

[그림 3-15] CareerSync 구직자 및 기업 이용 흐름도



(2) 비회원 채용공고 탐색 프로세스

CareerSync 는 비회원도 로그인 없이 공개된 채용공고를 탐색하고 상세 정보를 확인할 수 있도록 설계한다.

사용자는 메인 화면에서 채용공고 검색 및 필터링 기능을 이용하고, 원하는 채용공고의 상세 페이지에서 주요 업무, 자격 요건 및 기업 정보를 확인할 수 있다.

단, 입사 지원, 공고 저장, 이력서 업로드 및 AI 적합도 분석과 같은 개인화 기능을 이용하려는 경우에는 로그인이 필요하다. 로그인 후에는 기존에 확인하던 채용공고로 돌아가 해당 기능을 계속 이용할 수 있도록 설계한다.

[그림 3-16] 비회원 우선 채용공고 탐색 흐름도



(3) 구직자 채용공고 지원 프로세스

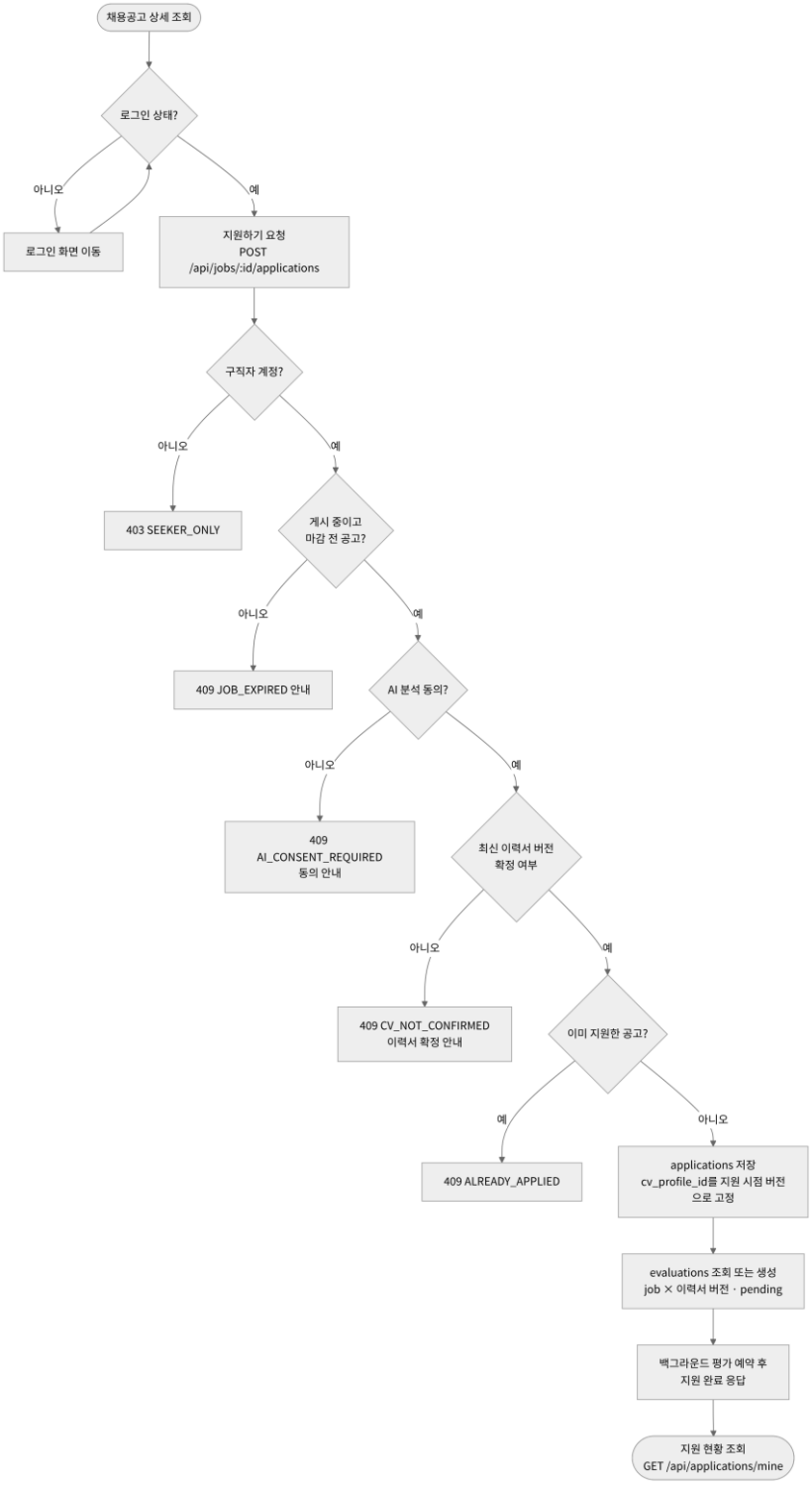
구직자는 메인 화면 또는 채용공고 탐색 화면에서 원하는 채용공고를 검색하고 상세 정보를 확인한다.

입사 지원을 요청하면 시스템은 로그인 상태를 확인한다. 비로그인 상태에서는 로그인 화면으로 이동하며, 로그인한 사용자는 등록된 이력서 정보를 확인한다.

이력서가 등록되지 않은 경우에는 이력서 등록을 안내한다. 지원에 필요한 정보가 준비되면 입사 지원 요청을 처리하고 지원 내역을 저장한다.

이후 구직자는 지원 현황 관리 화면에서 지원 결과와 진행 상태를 확인할 수 있도록 설계한다.

[그림 3-17] 구직자 채용공고 지원 프로세스 흐름도



지원 시 서버는 ① 구직자 계정 여부, ② 게시 중이며 마감되지 않은 공고인지, ③ AI 분석 동의 여부, ④ 가장 최신 이력서 버전의 확정 여부, ⑤ 중복 지원 여부를 순서대로 확인한다. 통과하면 지원 시점의 이력서 버전(cv_profile_id)을 고정하여 applications 에 저장하고, 평가 레코드(job × 이력서 버전)를

조회 또는 생성한 뒤 백그라운드 평가를 예약한다. 채용공고 목록·상세 조회(GET /api/jobs, GET /api/jobs/{job_id})는 비회원도 이용할 수 있으며, 지원하기를 선택하면 로그인 화면으로 이동한다.

(4) AI 기반 채용 적합도 분석 프로세스

구직자가 AI 채용 적합도 분석을 요청하면 시스템은 등록된 이력서와 분석 대상 채용공고의 데이터를 확인한다.

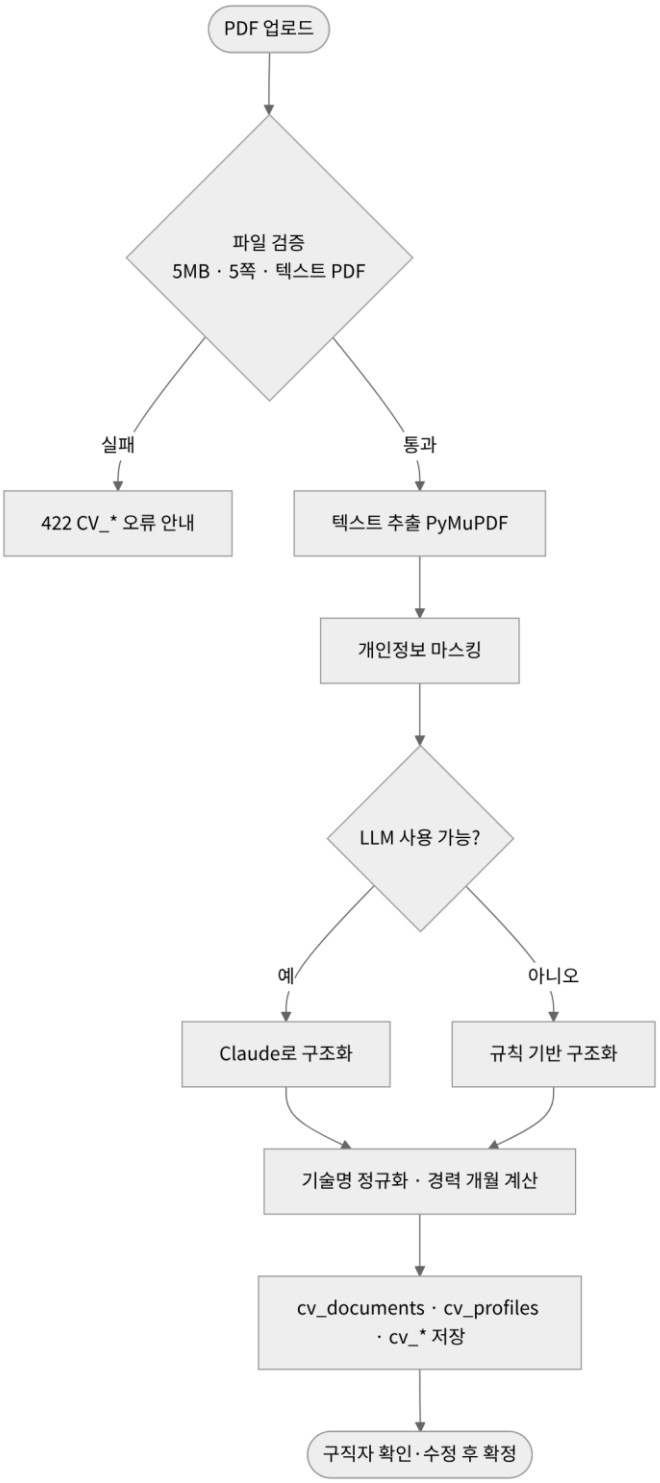
분석에 필요한 데이터가 정상적으로 존재하는 경우 텍스트 추출 및 전처리를 수행하고, 구직자의 보유 역량과 기업의 요구 조건을 추출한다.

이후 추출된 정보를 비교하여 적합도 점수, 일치하는 역량, 부족한 역량 및 추천 근거를 생성하고 사용자 화면에 제공하도록 설계한다.

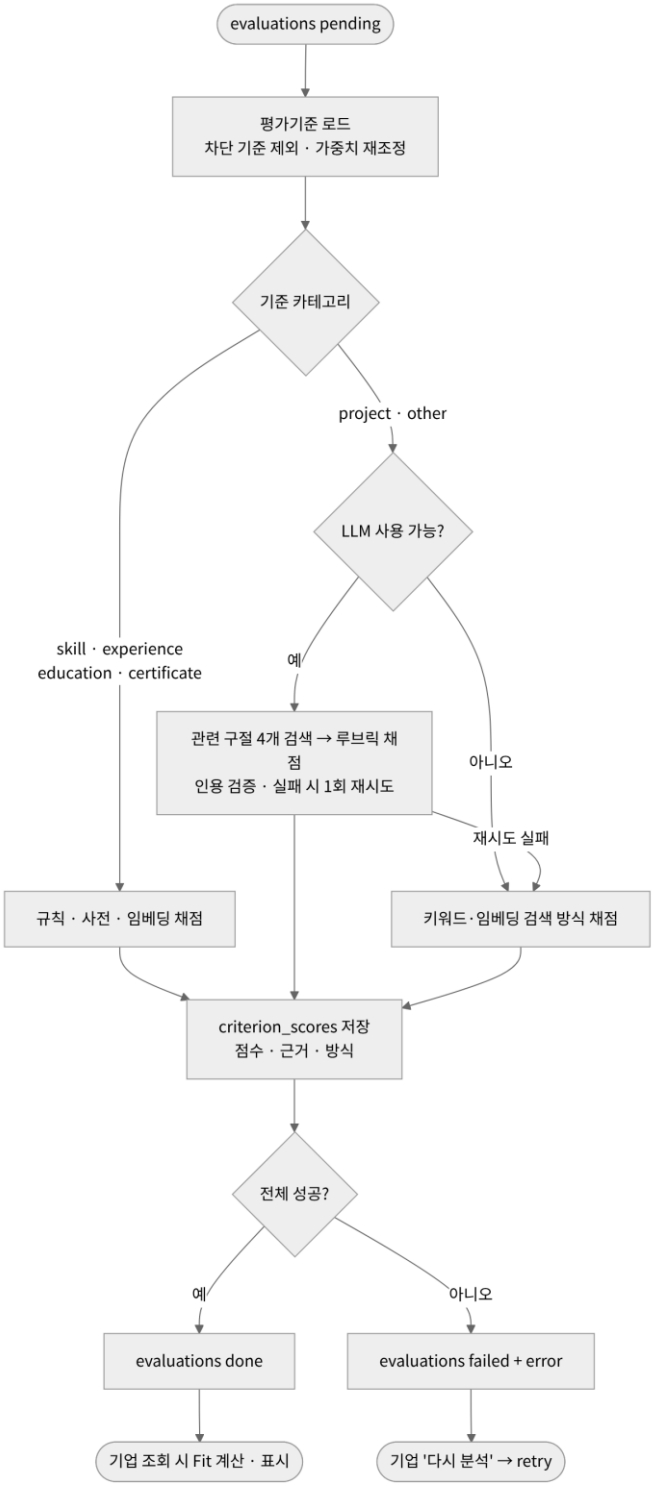
이력서 또는 채용공고 데이터가 누락되거나 분석 중 오류가 발생하면 사용자에게 해당 상태를 안내하고 필요한 조치를 수행할 수 있도록 한다.

[그림 3-18] AI 채용 적합도 분석 프로세스 흐름도

[그림 3-18-1] 이력서 업로드 및 역량 추출 단계



[그림 3-18-2] 평가기준별 적합도 계산 단계 (지원 후 백그라운드)



적합도 분석은 두 단계로 나뉜다. 이력서 업로드 시 텍스트 추출·전처리·역량 추출을 수행하여 구조화된 이력서를 저장하고, 입사 지원 후에는 백그라운드 작업이 공고의 평가기준별로 점수와 근거를 계산한다. 법적 가드레일에 의해 새로 차단된 기준은 채점에서 제외되고 나머지 기준의 가중치가 재조정된다.

(5) 기업 채용 관리 프로세스

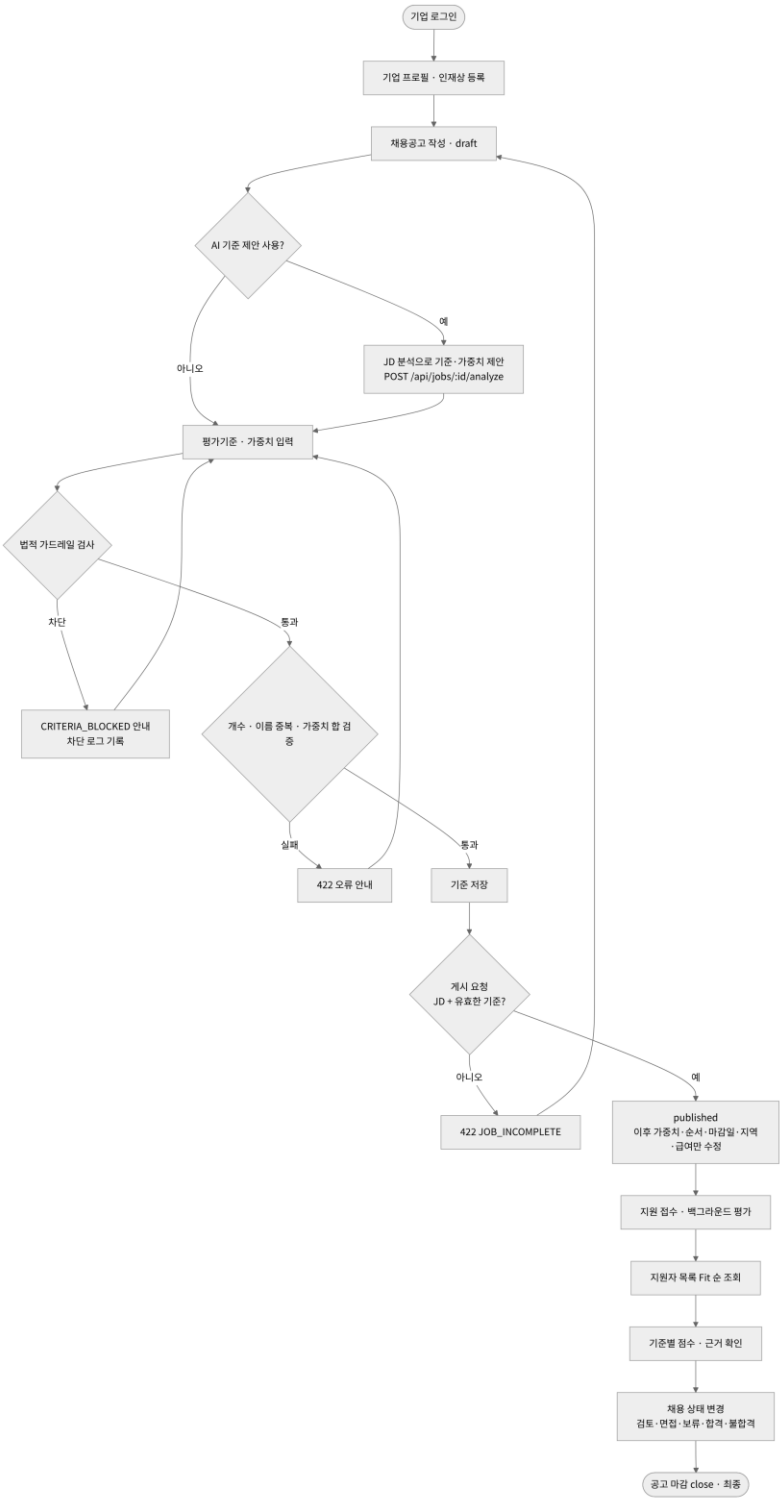
기업 사용자는 로그인 후 기업 대시보드에서 채용공고를 등록하고 관리한다.

채용공고 등록 시 직무명, 근무 조건, 필수 역량 및 우대 사항 등을 입력하고 필수 항목의 유효성을 확인한다.

입력 정보가 유효하지 않은 경우에는 수정 안내를 제공하며, 정상적으로 등록된 채용공고를 통해 지원자를 접수할 수 있도록 설계한다.

기업 담당자는 지원자 목록에서 이력서 정보와 AI 적합도 분석 결과를 확인하고, 검토 결과에 따라 채용 진행 상태를 변경하거나 면접 대상자를 선정할 수 있도록 한다.

[그림 3-19] 기업 채용 관리 프로세스 흐름도



채용공고는 draft → published → closed 순서로 관리한다. 게시하려면 JD(설명)와 유효한 평가기준이 필요하며, 평가기준은 저장 시 법적 가드레일 검사와 개수·이름 중복·가중치 합계 검증을 통과해야 한다. 게시 후에는 마감일·근무지·급여 안내와 기준의 가중치·순서만 변경할 수 있고, closed 는 최종 상태이다. 초안(draft)만 삭제할 수 있다.

4. 데이터베이스 기본설계 (Database Design)

4.1 주요 테이블 관계 정의표

구분	부모 테이블	자식 테이블	관계	설명
계정·이력서	**users**	cv_documents	1:N	한 구직자가 여러 이력서 문서를 등록
이력서 버전	**cv_documents**	cv_profiles	1:N	하나의 문서에 여러 구조화 이력서 버전을 저장
이력서 세부	**cv_profiles**	cv_skills / cv_experiences / cv_projects / cv_educations / cv_certificates	1:N	각 이력서 버전별 세부 데이터 분리 저장
프로젝트 기술	**cv_projects**	cv_project_techs	1:N	이력서 프로젝트별 사용 기술 저장
채용공고	**companies**	job_postings	1:N	기업별 여러 채용공고 등록
평가기준	**job_postings**	job_criteria	1:N	채용공고별 평가기준 설정
입사 지원	**job_postings**	applications	1:N	하나의 채용공고에 여러 지원 내역
지원 이력서	**cv_profiles**	applications	1:N	지원 당시 사용한 이력서 버전을 참조
AI 평가	**job_postings**	evaluations	1:N	공고별 지원자 이력서 평가 결과
평가 이력서	**cv_profiles**	evaluations	1:N	평가가 사용한 이력서 버전을 참조
세부 점수	**evaluations**	criterion_scores	1:N	평가 결과별 기준 점수 및 근거 저장
점수 기준	**job_criteria**	criterion_scores	1:N	평가기준별 여러 평가 점수 연결
구직자 선호	**users**	seeker_preferences	1:0..1	구직자 계정별 선호 설정 최대 1 건
선호 상세	**seeker_preferences**	seeker_preferred_locations / seeker_preferred_employment_types / seeker_job_keywords	1:N	희망 근무지·고용형태·직무 키워드를 별도 저장
기업 제안·프로젝트	**companies**	talent_offers / company_projects	1:N	기업별 인재 제안 및 프로젝트 관리

5. 인터페이스 설계 (Interface Design)

5.1 외부 인터페이스

외부 인터페이스는 Anthropic Claude API 하나이며, ai/llm/client.py 가 유일한 연결 지점이다 (Anthropic Python SDK 사용). 기본 모델은 claude-sonnet-5-5, effort 는 low 이다. LLM 에는 개인정보가 마스킹된 텍스트만 전달한다.

호출 기능	입력	출력	프롬프트
이력서 구조화	마스킹된 이력서 텍스트	cv_profile.v1 (기술·경력·프로젝트·학력·자격증·요약)	ai/prompts/v1/cv_structure.md
채용공고 기준 제안	채용공고 JD 텍스트	제안 평가기준 목록 (이름·카테고리·설명·가중치, 합계 100 으로 보정)	ai/prompts/v1/jd_analyze.md
project · other 기준 채점	기준 설명(루브릭) + 관련 이력서 구절 4 개 (+ other 는 기업 인재상)	점수(0/25/50/75/100) · 이유 · 인용 3 개 이하	ai/prompts/v1/

오류 발생 시 대체 처리

- API 키 미설정: LLM 을 호출하지 않고 규칙 기반 구조화 또는 키워드·임베딩 검색 방식 채점으로 처리한다.
- 인용 검증 실패: 1 회 재시도 후 검색 방식으로 대체한다. 평가 작업 실패 시 evaluations 를 failed 로 기록하고 재시도 경로를 제공한다.

환경변수 기반 연결 설정 (저장소 루트 .env)

변수	기본값	설명
LLM_API_KEY	없음	설정 시에만 Claude 를 호출한다 (유료)
LLM_MODEL / LLM_EFFORT	claude-sonnet-5-5 / low	사용 모델과 추론 강도
EMBEDDING_MODEL	intfloat/multilingual-e5-base	hashing-ngram-v1 지정 시 약 1GB 모델 다운로드 생략 (Compose 기본값)

※ 현재 코드는 ANTHROPIC_API_KEY 도 함께 읽으므로, 해당 변수가 설정된 환경에서는 의도치 않게 유료 호출이 발생할 수 있다 (알려진 이슈 1, 개선 예정).

5.2 내부 인터페이스

- 내부 인터페이스는 계정 인증(/api/auth), 사용자 관리(/api/users), 기업 정보(/api/companies), 채용공고(/api/jobs), 이력서(/api/cv), 입사 지원 및 평가 API 로 구성한다. 주요 요청·응답 항목은 상세설계 3 장에서 설명한다.

6. 사용자 인증 및 품질 요건

- 사용자 비밀번호는 Argon2id 방식으로 해시하여 저장하며, 로그인 후 발급받은 JWT Bearer 토큰으로 API 요청을 인증한다. 이력서·채용공고·지원자 데이터는 사용자 유형과 소유권에 따라 조회·수정 권한을 구분한다.

처리 시간

이력서 분석은 업로드 요청 안에서 동기로 처리되고, 지원자 평가는 지원 응답 후 백그라운드에서 처리되므로 지원 요청 자체는 평가 시간을 기다리지 않는다. LLM 을 사용하는 기준은 최대 4 개 스레드로 병렬 채점한다.

측정 항목	측정 구간	규칙 방식	LLM 방식
이력서 분석	POST /api/cv 요청 ~ 응답	측정 예정	측정 예정
지원자 평가	지원 완료 ~ evaluations done	측정 예정	측정 예정

처리 시간 목표치는 통합 테스트 담당이 1 차 측정한 실측값을 기준으로 설정한다.

실패 시 재시도 방식

- LLM 인용 검증 실패: 자동 1 회 재시도 후 검색 방식으로 대체한다.
- 평가 미완료(pending·failed): 기업이 '다시 분석'으로 POST /api/applications/{application_id}/evaluation/retry 를 호출한다. 이미 done 인 평가는 409 EVALUATION_DONE 을 반환한다.
- 제약 사항: 평가가 서버 프로세스 내부에서 실행되므로 서버가 재시작되면 pending 상태로 남으며 '다시 분석'이 필요하다. 또한 pending 상태에서 재시도하면 채점이 중복 실행될 수 있어 개선이 필요하다 (알려진 이슈 2, 6).

API 오류 응답

상황	HTTP 상태	응답 형식
업무 규칙 위반	404 · 409 · 422 등	{"code": "오류 코드", "message": "한국어 안내 메시지", ...}
요청 형식 검증 실패	422	{"detail": [...]}

토큰 없음·만료·변조	401	인증 오류
기업 전용 API 를 구직자가 호출	403	권한 오류 (구직자 전용 기능은 SEEKER_ONLY)
다른 기업의 공고 접근	404	JOB_NOT_FOUND (존재 여부를 노출하지 않기 위해 의도적으로 404)

테스트

pytest 를 저장소 루트에서 실행하며 DB·네트워크·API 키 없이 동작한다 (메모리 SQLite, hashing 임베더, FakeLLM). TEST_DATABASE_URL 로 별도 PostgreSQL 을 지정하면 PostgreSQL 에서도 실행할 수 있다. SQLite 테스트는 외래키 검사가 꺼져 있고 테이블을 마이그레이션이 아닌 create_all 로 생성하므로, 최종 검증은 PostgreSQL 환경에서 수행한다.

테스트 결과(테스트 수, 통과 여부, PostgreSQL 실행 결과)는 통합 테스트 담당이 기재한다.

7. AI 데이터 및 추론 설계

7.1 이력서 분석 및 데이터 처리 파이프라인

① 구직자가 PDF 이력서를 업로드하면 백엔드에서 파일 제약 검증 후 PyMuPDF 로 텍스트를 추출한다. ② PII 마스킹을 적용한다. ③ LLM(설정된 경우) 또는 규칙 기반으로 기술·경력·프로젝트·학력·자격 정보를 구조화한다. ④ cv_documents 와 cv_profiles 및 정규화 자식 테이블에 저장한다. ⑤ 구직자가 내용을 수정·확정하면 해당 문서의 새 버전으로 저장하고 이전 버전을 유지한다.

채용공고는 기업이 작성하고 평가기준(job_criteria)을 관리한다. 지원 시 확정된 이력서 버전이 applications.cv_profile_id 에 고정되며, 별도 평가 작업이 evaluations 및 criterion_scores 를 저장한다. 평가 완료 후 공고의 현재 가중치로 Fit 을 계산한다.

7.2 AI 모델 및 추론 구성

구성	주요 기술 및 처리 방식	설계 내용
이력서 PDF 파싱	PyMuPDF 기반 PDF 텍스트 추출, 개인정보 마스킹 및 이력서 항목 구조화	PDF 업로드와 분석 요청을 거쳐 구조화된 이력서 데이터 생성
LLM 구조화·평가	Claude Sonnet 계열 LLM 연동 및 규칙 기반 추출 방식	외부 모델 연결 상태에 따라 LLM 또는 규칙 기반 처리를 적용
의미 유사도	문장 임베딩 또는 n-gram 해싱 기반 벡터 표현	기술·직무 관련 표현의 의미적 유사도 비교에 활용

채용 평가	공고별 평가기준에 따라 항목별 점수·근거 기록	criterion_scores 에 항목별 결과를 저장하고 현재 가중치로 적합도 산출
양방향 추천 목록	구직자 선호 조건과 기업 채용 조건을 양방향으로 비교	추천 후보 필터링·정렬 및 근거 제공을 위한 인터페이스 설계

구성	입력	출력	설정 · 비고
이력서 정보 추출	마스킹된 이력서 텍스트	cv_profile.v1: 기술, 경력(기관·역할·시작/종료 YYYY-MM·설명·원문), 프로젝트(이름·역할·설명·사용 기술·원문), 학력(학교·전공·학위), 자격증(이름·취득 연월), 요약	Claude 또는 규칙. 기술명은 data/skills.yaml 별칭 사전으로 표준화, 경력 개월 수는 코드에서 계산
임베딩	기술명, 이력서 구절, 기준 설명	문장 벡터	intfloat/multilingual-e5-base (로컬 실행), 대체: hashing-ngram-v1
LLM 채점	기준 루브릭 + 관련 구절 4 개	점수 · 이유 · 인용 3 개 이하	claude-sonnet-5-5, effort low. project·other 기준만 사용

프롬프트 관리 방식

프롬프트는 ai/prompts/v1/ 아래에 버전별 Markdown 파일로 관리하고, 현재 버전은 ai/parsing/extract.py 의 PROMPT_VERSION 으로 지정한다. LLM 출력은 Pydantic 스키마(cv_profile.v1, jd_profile.v1)로 검증하며 이 스키마가 그대로 API 응답 본문이 된다. 평가마다 evaluations 에 llm_model, embedding_model, prompt_version, skills_version 을 기록하여 어떤 설정으로 계산된 결과인지 추적한다.

실패 시 대체 처리

API 키가 없으면 규칙 기반 추출과 검색 방식 채점을 사용한다. LLM 이 제시한 인용문은 마스킹된 이력서 원문에 실제로 존재하는지 검증하며, 검증 실패 시 1 회 재시도 후 검색 방식으로 대체한다.

분석 결과 예시 (가상 데이터)

```
기준: "백엔드 프로젝트 경험" (category: project, 가중치 30)
→ score: 75
  reason: "FastAPI 기반 REST API 를 설계·구현한 프로젝트 경험이 확인되나, 운영 환경 배포 경험은
```

```
확인되지 않음"
  evidence: ["FastAPI 로 회원·게시판 REST API 설계 및 구현"]
  method: "llm:claude-sonnet-5-5"
```

7.3 AI 분석 성능 평가

AI 분석 품질은 이력서 항목 추출의 정확도, 평가 근거의 적절성, 추천 순위의 적합도, 처리 시간 및 오류 발생 비율로 측정한다. 평가 항목별로 시험 데이터와 측정 방법을 정하여 검증 결과를 기록한다.

성능 평가는 실제 이력서가 아닌 합성 데이터로만 수행한다. 각 지표는 규칙 방식(LLM 미사용)과 LLM 방식을 나누어 측정하여 비교한다.

평가 항목	시험 데이터	지표	측정 방법	결과
이력서 항목 추출	합성 이력서 20 건 + 수작업 정답	기술·경력 항목 정밀도 / 재현율	추출 결과와 정답 비교	측정 예정
평가 근거 적절성	합성 이력서 × 공고 평가 결과	인용 검증 통과율, 근거 적절성(검토자 판정)	인용문 원문 존재 여부 + 수작업 검토	측정 예정
순위 적합도	공고 5 건 × 합성 이력서 20 건 + 수작업 순위	상위 5 명 일치율	Fit 순위와 수작업 순위 비교	측정 예정
처리 시간	위 데이터 전체	이력서 분석·평가 완료 시간	요청~응답, 지원~done 구간 측정	측정 예정
오류 발생 비율	위 데이터 전체	failed 평가 비율, 업로드 오류 비율	evaluations.status 집계	측정 예정

시험 데이터는 합성 이력서 20 건과 채용공고 5 건으로 구성하며, 정답(추출 항목·순위)은 수작업으로 작성한다. 지표 정의는 백엔드/AI 담당이, 측정 실행은 통합 테스트 담당이 수행하고, 측정 후 결과와 개선 사항을 기재한다.

8. 개인정보 및 서비스 데이터 관리

cv_documents 에는 업로드 PDF 의 파일명·해시·페이지 수·마스킹 텍스트·마스킹 유형을 기록한다. cv_profiles 는 분석 결과와 확정 시점만 기록하고 구조화 세부 항목을 하위 테이블에 보관한다. 사용자 역할(seeker/company)과 소유권 확인으로 이력서 조회·수정 권한을 제한한다.

구직자의 인재풀 공개 동의는 seeker_preferences 의 profile_public 및 profile_public_at 필드에 기록하고, 기업의 인재 제안 내역은 talent_offers 테이블에서 관리한다. 공개 프로필 제공 시에는 구직자 동의 상태를 확인하고, 제안 수락 후 연락처 제공은 별도의 권한 검증 절차를 거치도록 설계한다.

인재 추천 기능은 현재 데이터 구조(seeker_preferences.profile_public, talent_offers)가 준비되어 있으며, API 는 아래 기준으로 구현한다.

가입 시 AI 분석 동의 문구에 '지원 평가 및 맞춤 추천' 목적을 함께 명시하고 동의 버전(CONSENT_VERSION)을 갱신한다. 기업 측 인재 추천에는 인재풀 공개에 별도로 동의한(profile_public=true) 구직자만 노출한다.

기업이 볼 수 있는 익명 프로필

구분	항목
공개	익명 후보 ID, 보유 기술, 총 경력 기간, 경력의 역할(기관명 제외), 프로젝트 역할·사용 기술, 전공·학위, 자격증, 희망 근무지·고용형태·직무 키워드, 해당 공고 기준의 Fit 과 기준별 점수
비공개	이름(display_name), 이메일, 학교명, 경력 기관명, 이력서 원문 텍스트

노출 대상은 profile_public=true 인 구직자로 한정하고, 공개 동의 시점은 profile_public_at 에 기록한다.

인재 제안 수락·거절 절차

- 기업이 제안을 보내면 talent_offers 에 status=pending 으로 저장한다. 같은 기업이 같은 구직자에게 대기 중인 제안을 중복 생성하지 않도록 서비스에서 검사한다 (현재 DB 제약 없음).
- 구직자는 받은 제안 목록에서 수락(accepted) 또는 거절(rejected)하며, 응답 시각을 responded_at 에 기록한다. 기업은 pending 상태의 제안을 취소(cancelled)할 수 있다.

연락처 공개 시점

연락처는 제안이 accepted 상태가 된 이후에만 해당 기업의 제안 상세 응답에 포함한다. 원본 PDF 는 저장하지 않고 이력서 텍스트는 마스킹되어 있으므로, 공개 가능한 연락처는 users 테이블의 이름(display_name)과 이메일이다.

```
// 기업용 제안 상세 응답 예시 (가칭)
// pending · rejected · cancelled
{ "offer_id": 12, "status": "pending", "candidate": { "anonymous_id": "C-0042", "skills":
["Python", "FastAPI"] } }
// accepted
{ "offer_id": 12, "status": "accepted", "contact": { "display_name": "홍길동", "email":
"seeker@example.com" } }
```

AI 소프트웨어 상세설계서 (Detailed Design)

본 상세설계에서는 기본설계의 기능을 바탕으로 이력서 분석, 지원자 평가, 추천 기능, 데이터베이스 및 API 의 구성과 처리 절차를 구체화한다.

1. 컴포넌트 상세 설계

1.1 이력서 데이터 처리 모듈

절차	함수·모듈	입출력·처리 내용
파일 수신	POST /api/cv	UploadFile PDF → 파일 검증(최대 5MB, 텍스트 PDF 5 페이지 제한)
텍스트 추출	ai/parsing/pdf.py / extract_text()	PDF bytes → 추출 텍스트·페이지 수
비식별 처리	ai/parsing/pii.py / mask_pii()	텍스트 → masked_text, masked_kinds
구조화	ai/parsing/extract.py / structure_cv()	마스킹 텍스트 → CvProfile 데이터; LLM 또는 규칙 방식
DB 저장	cv_service.upload() / _fill_relational_profile()	cv_documents 1 건 + cv_profiles(version=1) 및 정규화 하위행
수정/확정	cv_service.update()	기존 프로필 유지 + version 증가하는 새 cv_profiles 행

API 에서는 구조화 결과를 CvOut.profile 객체로 반환하더라도 이는 직렬화 응답 형식이며, DB 에 JSONB 전체 프로필 컬럼으로 저장하는 구조가 아니다. DB 내부에서는 cv_profiles.summary 와 하위 관계형 테이블에 보관한다.

항목별 추출 규칙

항목	저장 위치	추출 규칙
기술	cv_skills	data/skills.yaml 의 별칭 사전으로 표준 기술명으로 정규화
경력	cv_experiences	기관·역할·시작/종료(YYYY-MM, 없으면 NULL)·설명·원문 줄. 경력 개월 수는 LLM 이 아닌 코드에서 계산
프로젝트	cv_projects, cv_project_techs	이름·역할·설명·원문 줄 + 사용 기술 목록
학력	cv_educations	학교·전공·학위·원문 줄 (채점에는 전공만 사용, 학교명은 채점하지 않음)
자격증	cv_certificates	이름·취득 연월(YYYY-MM)·원문 줄
요약	cv_profiles.summary	이력서 요약 문장

source(원문 줄)는 각 항목이 추출된 이력서의 줄을 기록한다. 구조화 방식은 cv_profiles.method 에 rules, llm:<모델명>, seeker(구직자 수정) 중 하나로 저장하고, 스키마 버전은 cv_profile.v1 이다.

모델 프롬프트

LLM 사용 시 ai/prompts/v1/cv_structure.md 프롬프트에 마스킹된 텍스트를 넣어 cv_profile.v1 형식의 JSON 을 요청하고, 결과를 Pydantic 스키마로 검증한다. API 키가 없으면 규칙 기반 추출을 사용한다.

예외 처리

상황	처리
5MB 초과 / PDF 아님 / 읽기 불가 / 암호화	422 CV_TOO_LARGE / CV_NOT_PDF / CV_UNREADABLE / CV_ENCRYPTED
5 쪽 초과 / 텍스트 없음(스캔 PDF)	422 CV_TOO_MANY_PAGES / CV_NO_TEXT
구직자가 아닌 계정	403 SEEKER_ONLY
LLM 미설정	규칙 기반 구조화로 처리

입력·출력 데이터 예시 (가상 데이터)

```
[입력: 마스킹된 텍스트 일부]
[NAME] / [EMAIL] / [PHONE]
2023.03 ~ 2024.02  ㅇㅇ테크 백엔드 인턴 - FastAPI 기반 사내 API 개발
프로젝트: 맛집 추천 서비스 (팀장) - Python, FastAPI, PostgreSQL
ㅇㅇ대학교 소프트웨어학부 학사
정보처리기사 (2024.06)

[출력: cv_profile.v1 요약]
skills: ["Python", "FastAPI", "PostgreSQL"]
experiences: [{org: "ㅇㅇ테크", role: "백엔드 인턴", start: "2023-03", end: "2024-02"}]
projects: [{name: "맛집 추천 서비스", role: "팀장", techs: ["Python", "FastAPI", "PostgreSQL"]}]
educations: [{school: "ㅇㅇ대학교", major: "소프트웨어학부", degree: "학사"}]
certificates: [{name: "정보처리기사", date: "2024-06"}]
```

1.2 지원 평가 및 후처리 모듈

application_service.apply()는 지원 공고 상태·동의·확정 이력서 존재 여부와 중복 지원 여부를 검사한다. 정상 지원 시 applications 를 생성하고 (job_id, cv_profile_id) 평가 레코드를 조회·생성한다. 필요한 경우 FastAPI BackgroundTasks 로 evaluation_service.run_in_background()를 실행한다.

evaluation_service.evaluate()는 job_criteria 의 기준별로 AI 결과를 산출하고 criterion_scores 에 score(0~100), level, reason, evidence, method 를 저장한다. evaluations 에 상태(pending/done/failed)와 model/prompt/skills 버전 및 실행 시각을 저장하며, Fit 은 현재 기준별 가중치와 저장된 점수를 사용하여 조회 시 계산한다.

평가기준별 점수 산정 (0~100)

카테고리	채점 방식	LLM
skill	기준에 명시된 기술과 이력서 기술 비교 (사전 + 임베딩 유사도). 알려진 기술명이 없으면 자유 텍스트로 검색 채점	사용 안 함
experience	총 경력 개월 수 vs 요구 연수 (기준 문장 → 없으면 공고의 min_experience_years → 없으면 2 년)	사용 안 함
education	전공만 평가 (필수 / 관련 / STEM). 학교명은 평가하지 않음	사용 안 함
certificate	기준에 명시된 자격증을 이력서에서 확인, '관련 자격증' 같은 일반 표현도 반영	사용 안 함
project, other	관련 이력서 구절 4 개 검색 → 루브릭 0/25/50/75/100 채점 + 인용 3 개 이하 → 인용 검증, 1 회 재시도. 키 없음·실패 시 키워드·임베딩 검색. other 는 기업 인재상을 함께 반영	키가 있을 때 사용

적합도(Fit) 계산

`Fit = Σ(가중치 × 점수) / Σ 가중치  (소수 첫째 자리 반올림, ai/scoring/fit.py)`

가중치는 criterion_scores 에 저장하지 않고 조회 시점의 현재 가중치를 사용하므로, 가중치를 바꾸면 AI 재호출 없이 순위가 다시 계산된다. 순위는 Fit 내림차순, 동점이면 STRONG_SCORE 이상인 기준 수가 많은 순, 그다음 먼저 지원한 순이다.

추천 근거 생성

각 기준 결과에는 reason(판단 이유), evidence(이력서 인용문), method(rule:skill, llm:<모델명>, retrieval 등)를 저장한다. LLM 인용문은 마스킹된 이력서에 실제로 있는 문장인지 검증한다. AI 결과를 포함하는 모든 응답에는 'AI 분석은 참고용입니다. 채용 여부는 기업이 최종 판단합니다.' 안내(AI_NOTICE)를 함께 제공한다.

계산 예시 (가상 데이터)

평가기준	카테고리	가중치	점수	가중치 × 점수
Python 활용 능력	skill	40	100	4,000

백엔드 경력 2 년 이상	experience	20	50	1,000
백엔드 프로젝트 경험	project	30	75	2,250
컴퓨터 관련 전공	education	10	100	1,000
합계		100		8,250 → Fit 82.5

1.3 추천·인재 제안 모듈

추천 기능은 구직자에게 적합한 채용공고를 제공하는 공고 추천과 기업에 적합한 구직자를 제공하는 인재 추천으로 구분한다. 구직자의 희망 근무 지역·고용형태·직무 키워드와 기업의 채용 조건을 후보 필터링에 활용하고, 관련 역량의 일치 정도 및 평가 근거를 포함해 추천 결과를 구성한다. 추천 조건과 제안 내역은 구직자 선호 테이블 및 인재 제안 테이블과 연계한다.

추천 기능은 현재 설계 단계이며(테이블만 존재), 핵심 원칙은 기업이 설정한 평가기준(job_criteria)을 지원자 평가와 추천에 동일하게 사용하는 것이다. evaluations 는 지원 건이 아닌 '공고 × 이력서 버전' 단위로 저장되므로 추천에서도 같은 평가 결과를 재사용한다.

공고 추천 (구직자 → 공고)

- 후보: 게시 중이며 마감 전인 공고
- 제외: 이미 지원한 공고, 확정 이력서가 없는 구직자, AI 분석 동의가 없는 구직자
- 구직자 표시: 기준별 점수는 보여주지 않고 적합도 구간(높음 Final ≥ 80 / 보통 60~79 / 낮음 40~59)과 일치 기준·부족 기준의 이름만 보여준다 (평가기준 가중치 대신 중요도만 공개하는 현재 방식과 동일).
- 점수: Final = 0.5 × Fit + 0.5 × Pref, Fit ≥ 40 이고 Pref ≥ 40 인 공고만 추천
- Fit: 지원자 평가와 동일한 방식으로 계산한 evaluations 를 재사용하고, 없으면 평가를 생성한다. 단, Pref ≥ 40 인 공고에 대해서만 Fit 을 계산하여 불필요한 LLM 호출을 줄인다.
- Pref: 희망 근무지, 희망 고용형태, 희망 직무 키워드(공고 제목·JD 와 비교) 3 개 항목을 동일 비중(각 1/3)으로 0~100 점 계산한다. 구직자가 입력하지 않은 항목은 일치로 간주한다. 근무지·직무 키워드는 자유 텍스트이므로 공백·대소문자 정규화 후 비교한다.

인재 추천 (기업 → 구직자)

- 후보: profile_public=true 이고 확정 이력서가 있는 구직자
- 제외: 해당 공고에 이미 지원한 구직자, 해당 기업의 대기 중 제안이 있는 구직자
- 점수: 같은 공고 평가기준으로 계산한 Fit 기준 정렬 (Pref 는 구직자 희망 조건과 공고 조건 일치도로 보조 반영)
- 응답: 익명 프로필 항목만 제공 (기본설계서 8 장 참조)

처리 순서

① 후보 필터링(조건·제외) → ② Pref 계산, Pref < 40 제외 → ③ 남은 후보의 evaluations 조회, 없으면 생성 → ④ Fit 계산(현재 가중치), Fit < 40 제외 → ⑤ Final 계산 → ⑥ 정렬 후 상위 N 건과 근거(일치 기준·부족 기준) 반환

API 요청·응답 형식 (가칭)

```
GET /api/recommendations/jobs?limit=20
→ 200 { "items": [ { "job_id": 7, "title": "백엔드 개발자", "match_level": "보통",
         "matched": ["Python 활용 능력"], "gaps": ["백엔드 경력 2 년 이상"] } ], "notice":
AI_NOTICE }

GET /api/jobs/{job_id}/talent?limit=20   (기업 전용)
→ 200 { "items": [ { "anonymous_id": "C-0042", "fit": 80.0, "skills": [...] } ], "notice":
AI_NOTICE }
```

2. 데이터베이스 상세설계

2.1 환경·스키마 구성 및 저장 원칙

DBMS 는 PostgreSQL 16 이며 Python 백엔드는 SQLAlchemy ORM 과 Alembic 마이그레이션을 이용한다. 주요 테이블은 정수형 id 를 기본키로 사용한다. 이력서 데이터는 문서 메타데이터·개인정보 마스킹 텍스트(cv_documents), 이력서 버전·요약(cv_profiles), 기술·경력·프로젝트·학력·자격증 세부 테이블로 구분하여 관리한다. 전체 분석 결과를 단일 JSONB 컬럼에 저장하지 않는다.

테이블 생성·변경은 Alembic 마이그레이션으로 관리한다. 이력서 등록과 수정, 채용공고 지원, 평가 결과 저장 기능은 SQLAlchemy 관계형 모델을 이용해 참조 무결성과 일관된 데이터 처리를 유지한다.

2.2 물리 테이블 정의(총 23 개)

물리 테이블	주요 컬럼(PK/FK 및 핵심 속성)	용도 / 제약
users	id PK, email, password_hash, display_name, role, company_id FK, consent_*	개인·기업 계정; role=seeker/company; 소문자 이메일 유니크 인덱스
companies	id PK, name, intro, talent_profile, created_at, updated_at	기업 기본 프로필
job_postings	id PK, company_id FK, title, description, employment_type, min_experience_years, location, deadline, status	공고 1:N; draft/published/closed
job_criteria	id PK, job_id FK, name, description, category, weight, position, guardrail_version	공고별 채용 평가기준 및 가중치
criteria_guardrail_logs	id PK, company_id FK, job_id FK(선택), input_name, rule_id, category, law_ref, blocklist_version	평가기준 차단 감사로그
cv_documents	id PK, seeker_id FK, file_name, sha256, pages, masked_text, masked_kinds, created_at	파일 메타데이터·개인정보 마스킹 텍스트
cv_profiles	id PK, cv_document_id FK, seeker_id FK, version, schema_version, method, confirmed_at, summary	이력서 문서별 버전; (cv_document_id, version) UNIQUE
cv_skills	id PK, cv_profile_id FK, skill_name	이력서 버전별 기술 목록
cv_experiences	id PK, cv_profile_id FK, org, role, start, end, description, source	이력서 버전별 경력
cv_projects	id PK, cv_profile_id FK, name, role, description, source	이력서 버전별 프로젝트
cv_project_techs	id PK, cv_project_id FK, tech_name	특정 이력서 프로젝트의 기술
cv_educations	id PK, cv_profile_id FK, school, major, degree, source	이력서 버전별 학력
cv_certificates	id PK, cv_profile_id FK, name, date, source	이력서 버전별 자격증
applications	id PK, job_id FK, seeker_id FK, cv_profile_id FK, status, status_updated_at	지원과 지원 시점 확정 이력서 버전
evaluations	id PK, job_id FK, cv_profile_id FK, status, error, llm_model, embedding_model, prompt_version, skills_version	공고×이력서 버전 단위 평가
criterion_scores	id PK, evaluation_id FK, job_criteria_id FK, score, level, reason, evidence, method	항목별 평가 점수·근거; evidence JSONB 유지
seeker_preferences	id PK, seeker_id FK, profile_public, profile_public_at, created_at, updated_at	구직자 공개 동의와 선호 조건의 상위행
seeker_preferred_locations	id PK, preference_id FK, location	복수 희망 근무 지역
seeker_preferred_employment_types	id PK, preference_id FK, employment_type	희망 고용형태(full_time/contract/intern)
seeker_job_keywords	id PK, preference_id FK, keyword	희망 직무 키워드
talent_offers	id PK, company_id FK, seeker_id FK, job_id FK(선택), status, created_at, responded_at	기업 인재제안·응답 상태
company_projects	id PK, company_id FK, name, description, created_at, updated_at	기업 프로젝트 이력
company_project_techs	id PK, company_project_id FK, tech_name	기업 프로젝트별 기술

아래 표는 각 테이블의 식별자와 핵심 속성을 중심으로 정리한 것이다. 날짜·갱신 시각·기본값 등 부가

속성은 일부 생략했으며, 컬럼 자료형과 상세 제약은 데이터베이스 스키마 정의에 따른다. 가변적인 자료 구조는 cv_documents.masked_kinds 와 criterion_scores.evidence 등 제한된 필드에서 JSONB 형식으로 관리한다.

2.3 핵심 관계 및 참조 무결성

부모 → 자식	카디널리티	참조 기준·삭제 정책
users → cv_documents → cv_profiles	1:N → 1:N	cv_document_id 기준 버전 연결; 문서 삭제 시 프로필 CASCADE
cv_profiles → cv_skills/experiences/projects/educations/certificates	1:N	각 자식은 cv_profile_id FK, 프로필 삭제 시 CASCADE
cv_projects → cv_project_techs	1:N	cv_project_id FK, 프로젝트 삭제 시 CASCADE
companies → job_postings → job_criteria	1:N → 1:N	company_id/job_id FK; 게시공고의 평가기준 관리
job_postings·users·cv_profiles → applications	각 부모 1:N	(job_id,seeker_id) UNIQUE; cv_profile_id 삭제 RESTRICT
job_postings·cv_profiles → evaluations → criterion_scores	각 부모 1:N	(job_id,cv_profile_id) UNIQUE; score 0~100; 평가 세부행 CASCADE
users → seeker_preferences → 세 종류 선호 테이블	1:0..1 → 1:N	seeker_id UNIQUE; (preference_id,값) UNIQUE
companies·users → talent_offers	각 부모 1:N	job_id 선택; 채용공고 삭제 시 SET NULL
companies → company_projects → company_project_techs	1:N → 1:N	기술명은 프로젝트 안에서 중복 불가

2.4 저장·조회·버전 관리 시퀀스

- 신규 업로드: PDF 검증 → 원문 텍스트 추출·PII 마스킹 → cv_documents INSERT → cv_profiles(version=1, summary 등) INSERT → cv_* 하위행 저장 → COMMIT.
- 수정/확정: 특정 cv_profiles.id 에 대한 소유권 검증 → 동일 cv_document_id 의 MAX(version)+1 생성 → 변경 프로필의 하위행 신규 저장 → confirmed_at 선택적으로 기록 → COMMIT. 기존 버전은 UPDATE 하지 않음.
- 지원: 최신 확정본을 선택 → applications.cv_profile_id 를 고정 → (job_id, seeker_id) 중복 UNIQUE 및 기존 평가 (job_id, cv_profile_id) 재활용 → 완료 시 기준별 점수를 조회.
- 평가기준 변경: criterion_scores 에는 기준별 원점수를 저장하고, 가중치 반영 Fit 은 조회 시 계산하므로 변경된 가중치를 기존 점수에 적용 가능.
- 보호: 직접 로그인 사용자의 이력서 API 는 cv_profiles.seeker_id 와 현재 사용자 id 를 비교하여 타인 버전 접근을 막음.

2.5 제약조건·인덱스·상태값

주요 UNIQUE: cv_profiles(cv_document_id,version), applications(job_id,seeker_id),

evaluations(job_id,cv_profile_id), criterion_scores(evaluation_id,job_criteria_id), seeker_preferences(seeker_id), 선호 값 중복 방지 인덱스. 자주 조인·검색하는 FK 에는 ix_* 인덱스가 있으며 이메일은 lower(email) 기준 유일 인덱스가 정의된다.

주요 CHECK: users.role=seeker/company; applications.status=submitted/reviewing/interview/on_hold/passed/rejected; evaluations.status=pending/done/failed; criterion_scores.score 0~100; job_criteria.weight 1~100; talent_offers.status=pending/accepted/rejected/cancelled; 희망 고용형태 full_time/contract/intern. 상태 변경은 업무 권한 검증과 함께 수행한다.

2.6 이력서 데이터 정규화 및 스키마 변경

리비전	실제 작업	확인 포인트
0004	cv_documents, cv_profiles(profile JSONB), applications, evaluations, criterion_scores 추가	초기 구조
0005	cv_profiles.summary 추가 + 6 개 CV 정규화 하위 테이블 생성	이전 과정에서는 구 JSON 열 유지
0006	기존 cv_profiles.profile JSONB 값을 하위 테이블에 백필하고 profile 컬럼 삭제	이관 후 데이터 검증 필수; downgrade 복원 로직 있음
0007	구직자 선호 4 개, 인재 제안 1 개, 기업 프로젝트 2 개 테이블 추가	데이터 구조만 생성, 추천·인재풀 API 별도 개발 필요

이력서 저장 구조는 JSONB 중심의 초기 구조를 관계형 구조로 전환하도록 설계하였다. 변경 단계에서는 새로운 세부 테이블을 생성한 다음 기존 JSONB 의 내용을 관계형 테이블로 이전하고, 이관 확인 후 기존 전체 프로필 JSONB 컬럼을 제거한다. 데이터 마이그레이션 시에는 백업, 이전 전후 건수 비교, 외래키 유효성 검사 및 오류 발생 시 복구 절차를 함께 수행한다.

2.7 개인정보 공개 동의 및 접근 정책

seeker_preferences.profile_public 와 profile_public_at 는 인재풀 공개 여부와 동의 시점을 저장한다. talent_offers 는 인재 제안을 보낸 기업·구직자·관련 공고 및 수락 상태를 관리한다. 익명 프로필의 조회와 연락처 공개는 해당 테이블의 동의·수락 정보를 백엔드 API 에서 검증한 후 허용하도록 설계한다.

채용공고에 직접 지원한 구직자의 지원 정보와 기업의 인재풀 탐색 대상은 구분하여 관리한다. 인재풀을 통한 후보자 탐색에서는 별도 공개 동의와 제안 상태에 따라 제공 정보를 제한한다.

2.8 전체 테이블 관계 상세 명세표

영역	테이블명 (PK: id)	FK 및 참조 대상	관계 / 주요 제약
계정·기업	**companies**	—	기업 정보의 기준 테이블
계정·기업	**users**	company_id → companies.id (선택)	companies 1:N users; 기업 사용자에게 company_id 필요
채용공고	**job_postings**	company_id → companies.id	companies 1:N job_postings
채용공고	**job_criteria**	job_id → job_postings.id	job_postings 1:N job_criteria; 동일 공고 내 기준명 중복 금지
채용공고	**criteria_guardrail_logs**	company_id → companies.id job_id → job_postings.id (선택)	companies 1:N logs; job_postings 1:N logs (선택 연결)
이력서	**cv_documents**	seeker_id → users.id	users 1:N cv_documents; 마스킹 텍스트 저장
이력서	**cv_profiles**	cv_document_id → cv_documents.id seeker_id → users.id	각 부모 1:N cv_profiles; (cv_document_id, version) UNIQUE
이력서	**cv_skills**	cv_profile_id → cv_profiles.id	cv_profiles 1:N cv_skills
이력서	**cv_experiences**	cv_profile_id → cv_profiles.id	cv_profiles 1:N cv_experiences
이력서	**cv_projects**	cv_profile_id → cv_profiles.id	cv_profiles 1:N cv_projects
이력서	**cv_project_techs**	cv_project_id → cv_projects.id	cv_projects 1:N cv_project_techs
이력서	**cv_educations**	cv_profile_id → cv_profiles.id	cv_profiles 1:N cv_educations
이력서	**cv_certificates**	cv_profile_id → cv_profiles.id	cv_profiles 1:N cv_certificates
지원·평가	**applications**	job_id → job_postings.id seeker_id → users.id cv_profile_id → cv_profiles.id	각 부모 1:N applications; (job_id, seeker_id) UNIQUE; 이력서 참조 삭제 제한
지원·평가	**evaluations**	job_id → job_postings.id cv_profile_id → cv_profiles.id	각 부모 1:N evaluations; (job_id, cv_profile_id) UNIQUE
지원·평가	**criterion_scores**	evaluation_id → evaluations.id job_criteria_id → job_criteria.id	각 부모 1:N criterion_scores; (evaluation_id, job_criteria_id) UNIQUE
구직자 선호	**seeker_preferences**	seeker_id → users.id	users 1:0..1 seeker_preferences; seeker_id UNIQUE
구직자 선호	**seeker_preferred_locations**	preference_id → seeker_preferences.id	seeker_preferences 1:N; (preference_id, location) UNIQUE
구직자 선호	**seeker_preferred_employment_types**	preference_id → seeker_preferences.id	seeker_preferences 1:N; (preference_id, employment_type) UNIQUE
구직자 선호	**seeker_job_keywords**	preference_id → seeker_preferences.id	seeker_preferences 1:N; (preference_id, keyword) UNIQUE
인재 제안	**talent_offers**	company_id → companies.id seeker_id → users.id job_id → job_postings.id (선택)	각 참조 부모 1:N offers; job_id 는 NULL 허용
기업 프로젝트	**company_projects**	company_id → companies.id	companies 1:N company_projects
기업 프로젝트	**company_project_techs**	company_project_id → company_projects.id	company_projects 1:N techs; (company_project_id, tech_name) UNIQUE

3. API 인터페이스 상세 명세

3.1 주요 API 목록

HTTP / 경로	주요 입출력·권한	관련 모듈
POST /api/auth/register · /api/auth/login	회원 가입 / JWT 토큰 발급	api/v1/auth.py
GET /api/users/me	인증 사용자 정보	api/v1/users.py
GET·PUT /api/companies/me	기업 프로필 조회·수정	api/v1/companies.py
GET·POST /api/jobs, GET /api/jobs/{job_id}	공고 검색·등록·상세 조회	api/v1/jobs.py
POST /api/jobs/{job_id}/analyze	채용공고 내용 AI 분석 제안	api/v1/jobs.py
PUT/POST /api/jobs/{job_id}/criteria 등	평가기준 등록·수정·검증·차단 기록	api/v1/criteria.py
POST /api/cv	PDF 업로드 → CvOut 반환 (구직자만)	api/v1/cv.py
GET /api/cv/me, GET /api/cv/{profile_id}	최신 이력서 / 특정 버전 본인 조회	api/v1/cv.py
PUT /api/cv/{profile_id}	CvUpdate(profile, confirm) → 새 버전 CvOut	api/v1/cv.py
POST /api/jobs/{job_id}/applications	지원 및 AI 평가 예약	api/v1/applications.py
GET /api/applications/mine	본인 지원 내역	api/v1/applications.py
GET /api/jobs/{job_id}/applications	기업 지원자 순위·Fit	api/v1/applications.py
GET /api/applications/{id}/evaluation	기업 평가 결과·근거	api/v1/applications.py
POST /api/applications/{id}/ evaluation/retry	평가 재실행	api/v1/applications.py
PUT /api/applications/{id}/status	기업 담당자 상태 변경	api/v1/applications.py

아래 API 는 기능별로 사용자 인증, 채용공고 관리, 이력서 관리, 입사 지원 및 평가로 구분한다. 상세한 요청 및 응답 형식과 오류 처리 명세는 기능별 API 정의에서 기술한다.

모든 API 는 /api 접두사를 사용하며, register·login·/health 와 비회원 공개 조회(GET /api/jobs, GET /api/jobs/{job_id}, 게시 중인 공고만)를 제외하고 Authorization: Bearer <token> 이 필요하다(토큰 유효시간 JWT_EXPIRE_MINUTES, 기본 120 분). 필드 단위의 요청·응답 스키마는 서버의 Swagger 문서(/docs)를 기준으로 한다.

주요 API 상태코드 및 오류 코드

API	성공	주요 오류
POST /api/auth/register	가입 완료	409 EMAIL_TAKEN, 422
POST /api/auth/login	토큰 발급	401 INVALID_CREDENTIALS
POST /api/jobs	초안 생성	422 DEADLINE_IN_PAST
PATCH /api/jobs/{job_id}	수정	409 JOB_CLOSED, JOB_FIELD_LOCKED · 422 FIELD_REQUIRED
DELETE /api/jobs/{job_id}	초안 삭제	409 JOB_NOT_DRAFT
POST /api/jobs/{job_id}/publish	게시	409 JOB_NOT_DRAFT · 422 JOB_INCOMPLETE
POST /api/jobs/{job_id}/close	마감	409 JOB_NOT_PUBLISHED
PUT /api/jobs/{job_id}/criteria	기준 저장	422 CRITERIA_COUNT_INVALID, CRITERIA_DUPLICATE_NAME, WEIGHT_SUM_INVALID, CRITERIA_BLOCKED · 409 CRITERIA_LOCKED
POST /api/cv	CvOut	422 CV_TOO_LARGE, CV_NOT_PDF, CV_UNREADABLE, CV_ENCRYPTED, CV_TOO_MANY_PAGES, CV_NO_TEXT
GET /api/cv/me	CvOut	404 CV_NOT_FOUND
POST /api/jobs/{job_id}/applications	지원 완료	409 JOB_EXPIRED, AI_CONSENT_REQUIRED, CV_NOT_CONFIRMED, ALREADY_APPLIED
GET /api/applications/{id}/evaluation	기준별 점수·이유·근거	404 APPLICATION_NOT_FOUND
POST /api/applications/{id}/evaluation/retry	재평가 시작	409 EVALUATION_DONE
공통		401 토큰 오류 · 403 권한 없음(구직자 전용은 SEEKER_ONLY) · 404 JOB_NOT_FOUND(타 기업 공고 포함)

오류 응답 예시

```
HTTP 409
{ "code": "CV_NOT_CONFIRMED", "message": "(한국어 안내 메시지)" }

HTTP 422 (요청 형식 오류)
{ "detail": [ { "loc": [...], "msg": "...", "type": "..." } ] }
```

추가 예정 API (경로는 가칭, 요청·응답 형식 확정 전)

기능	API (가칭)	관련 테이블
구직자 선호 조건·인재풀 공개 조회/수정	GET · PUT /api/preferences/me	seeker_preferences 외 3 개
공고 추천	GET /api/recommendations/jobs	evaluations, seeker_preferences
인재 추천 (익명 목록)	GET /api/jobs/{job_id}/talent	seeker_preferences, evaluations
인재 제안 보내기 / 취소	POST /api/talent-offers · POST /api/talent-offers/{id}/cancel	talent_offers
받은 제안 조회 / 수락 / 거절	GET /api/talent-offers/mine · POST /api/talent-offers/{id}/accept · /reject	talent_offers
기업 프로젝트 관리	GET · POST · PUT · DELETE /api/companies/me/projects	company_projects, company_project_techs

추가 API 의 경로는 위 표와 같이 정하며, 필드 단위 요청·응답 형식은 구현 시 Swagger(/docs)에 반영한다. 또한 채용공고 목록·상세 조회를 비회원에게 공개하도록 GET /api/jobs, GET /api/jobs/{job_id}의 인증 조건을 변경한다(게시 중·마감 전 공고만 노출).

3.2 DB 연동에 관한 API 불변조건

POST /api/cv 는 cv_documents 와 cv_profiles 및 자식 테이블을 하나의 저장 흐름으로 기록한다. PUT /api/cv/{profile_id}는 기존 행을 덮어쓰지 않고 새로운 버전을 저장한다. POST /api/jobs/{job_id}/applications 는 확정 프로필이 없으면 거절하고 applications.cv_profile_id 를 지원 시점으로 고정한다. 평가 응답은 criterion_scores 와 현재 job_criteria 가중치의 조합에서 생성한다.

4. 시퀀스 다이어그램 및 비동기 처리

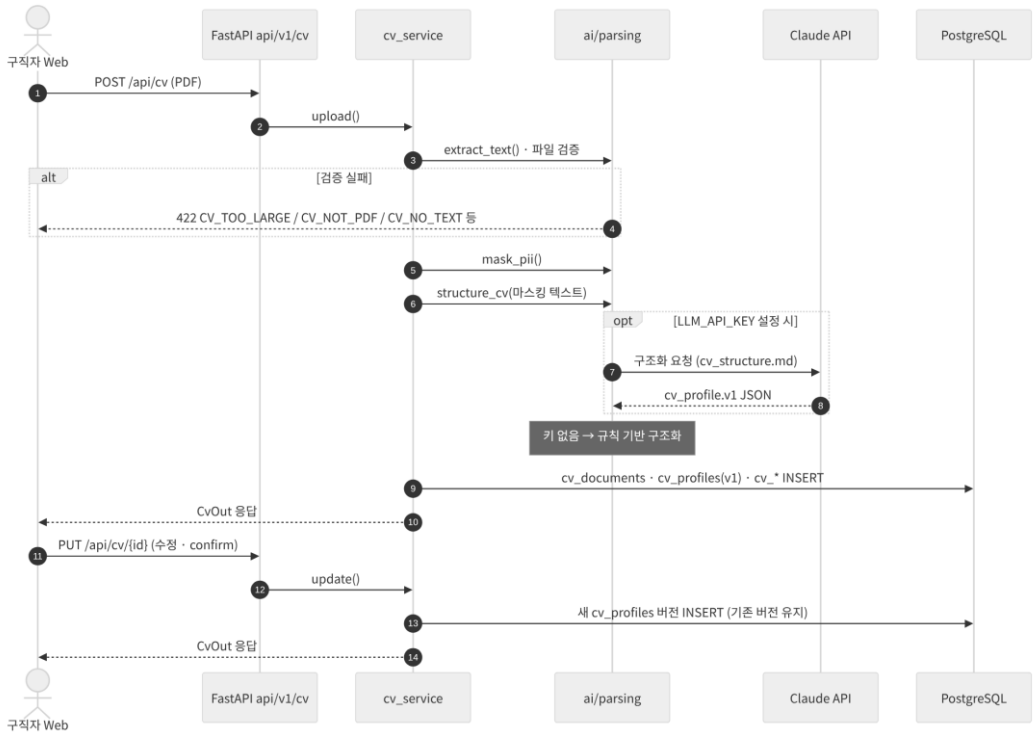
4.1 이력서 업로드 및 분석 과정

Web → POST /api/cv → pdf.extract_text → mask_pii → structure_cv(LLM 또는 규칙 대체) → DB(cv_documents, cv_profiles, cv_*) → CvOut 응답. 신규 프로필 편집 시 PUT /api/cv/{id} → DB 새 버전 → CvOut 응답.

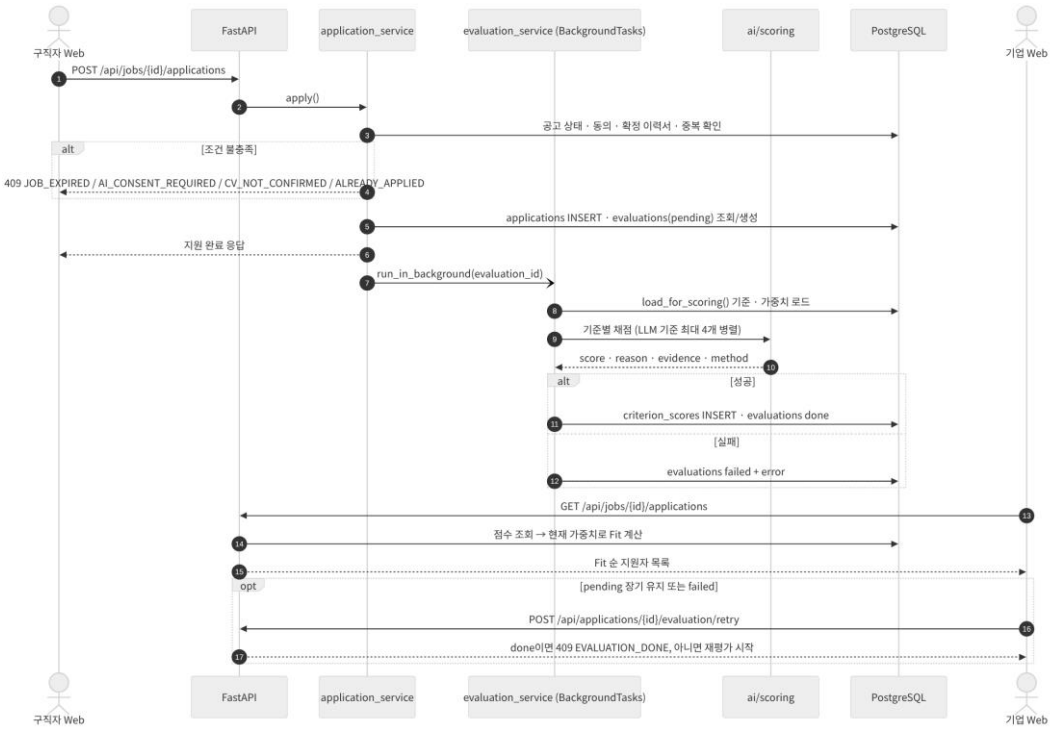
4.2 입사 지원 및 평가 과정

Web → POST /api/jobs/{job_id}/applications → 소유·동의·확정 버전 검사 → DB(applications, evaluations pending) → FastAPI BackgroundTasks → 평가 기준별 AI 추론 → DB(criterion_scores, evaluations done/failed) → 기업이 GET 지원자 목록/평가로 Fit 과 근거를 확인.

[그림 4-1] 이력서 업로드 및 분석 시퀀스



[그림 4-2] 입사 지원 및 비동기 평가 시퀀스



비동기 평가 실패·재시도 처리 흐름

- 채점 중 오류가 발생하면 evaluations.status=failed 와 error 를 저장하고, 기업 화면에 '다시 분석' 버튼을 표시한다.
- 재시도 요청 시 done 이면 409 EVALUATION_DONE 을 반환하고, 그 외에는 평가를 다시 실행한다.
- 서버가 재시작되면 실행 중이던 평가는 pending 으로 남으므로 재시도가 필요하다. pending 상태 재시도 시 채점이 중복 실행될 수 있어 개선이 필요하다 (알려진 이슈 2, 6).
- 중복 실행 방지: 재시도는 failed 상태이거나 pending 상태로 10 분 이상 경과한 평가에만 허용하고, 그 외에는 409 로 거절한다.

5. 예외 처리 및 테스트 설계

영역	검증 내용	기대 결과 및 기준
PDF 검증	PDF 의 용량·페이지·텍스트 유효성 검사	형식이나 크기 제한을 초과한 파일은 오류 상태를 반환
소유권·역할	사용자 인증 및 이력서·공고 소유권 검사	권한 없는 정보 열람 및 변경 요청은 거부
이력서 무결성	이력서 버전별 외래키와 확정 이력서	참조 무결성이 유지되고 지원 당시

영역	검증 내용	기대 결과 및 기준
	지원 검증	이력서 버전이 보존
평가 상태	지원자 평가의 pending/done/failed 상태 관리	정상 처리 시 점수·근거 저장, 실패 시 오류 상태와 재시도 경로 유지
개인정보 공개	공개 동의와 인재 제안 수락 상태 확인	동의 없는 인재풀 공개를 제한하고 승인된 범위만 제공
AI 성능	이력서 추출 및 추천 품질과 처리 지연 평가	합의한 시험 데이터와 지표로 측정 결과 기록